Section 1: About You

Goals and Objectives

<table>
<tr><td>Why do you want to do a GSoC project with Oppia?</td><td colspan="2">As I delved into the world of open-source projects, I found Oppia, which resonated strongly with my skill set in Python and TypeScript. I was also drawn to Oppia's mission of providing free education to everyone. Initially, my goal was to refine my skills through contributions to open-source projects. However, as I became more involved, I found it a very exciting process and the continual learning it integrated. Plus, the people of Oppia are super helpful to newcomers like me, which has made it even more enjoyable.

Participating in GSoC will be the perfect opportunity for me to upskill and engage in meaningful work. My experience working with the LaCe team made me familiar with the codebase and interested in knowing how web app abstraction works along with their backend code. I firmly believe that GSoC will provide me with insightful learning and growth.</td></tr>
<tr><td>What do you hope to learn/achieve during GSoC?</td><td colspan="2">In this journey, I have confidence that I will undergo the following upskilling-:

● Efficient way to deal with problems- hope to learn shorter and quicker ways to address a problem, and write good quality and maintainable code.

● Time management- It will also be a learning experience for me concerning delivering tasks undertaken under time-bound conditions.

● Increase in knowledge- getting familiarized with other standard software development practices in the industry.

● Debugging- I will improve my debugging skills and be able to fix errors more efficiently and effectively.</td></tr>
<tr><td rowspan="4">Which Oppia teams have you collaborated with, and what have you done on those teams?</td><td>LaCE Team</td><td>● Led the LaCE team and helped in the team management.
● Helped with the triage process on many issues.
● Helped the teammates with issues.</td></tr>
<tr><td>Angular Migration Team</td><td>● Migrated many pages from AngularJS to Angular2+.
● Collaborated with senior devs for the completion of angular migration</td></tr>
<tr><td>Welfare Team</td><td>● Helped to onboard new contributors in discussions.</td></tr>
<tr><td>Release Team</td><td>● Took part in release testing (3.3.5 - 3.3.8) as a manual tester.
● Took part in release testing (3.3.9 - 3.4.5) as a regression tester.
● I filed many of CUJ's issues during the release testing.</td></tr>
</table>

Contact information	Contact Gmail/Google Chat: hardikgoyal2003@gmail.com Optional Gmail: hardikgoyal1503@gmail.com LinkedIn: Hardik Goyal
Preferred method of communication	Email, Google chat
Social media handles (optional) *This field is **optional.** We ask for it so that we can tag you in GSoC-related posts if your application is accepted!*	LinkedIn: Hardik Goyal
Which timezone will you primarily be in during the summer?	Indian Standard Time(IST) or GMT + 5:30
(If you are a student) When are your school holidays?	My end-semester exams are expected to finish by 9th June (assuming no delays). After that, I will have holidays until 1st August (tentatively). During this break, I can easily dedicate 30+ hours per week or more as needed for projects.
What other obligations might you need to work around during the summer?	Currently, I am in my 6th semester, and based on the university's announced datesheet, my end-semester exams are scheduled between 26th May and 9th June. However, there is a possibility of a slight delay, and exams might actually begin in the 1st week of June instead of the last week of May. Out of all subjects, I anticipate needing about 2 days each to prepare for Control Systems (scheduled on 6th June) and Antenna & Wave Propagation (scheduled on 9th June), since those require extra attention. I've already started preparing for these two, but I'm keeping this buffer in case I need additional revision time. The other subjects are in my strong areas and won't require much prep. Post exams, I don't have any significant academic obligations. After 1st August, college will resume, but I'll only be required to attend once a week or once every two weeks for about 3–4 hours, so my schedule will remain flexible.
Planned time commitment	I will be dedicating 30+ hours per week to my work on this project and willing to extend my hours if required. Detailed Overview: • I will work 2-3 hours a day until 10 June as my exams end (Tentative). • I will work 6 hours a day, 6 days/week starting from 11th June - 1st August (Tentative). • I will work 5–6 hours a day, 6 days/week starting from August 1st. I may need to attend college either once a week for 3–4 hours or once

	every two weeks for the same duration to give presentations on my final year project.

Project Timeframe

Note: *Oppia will only be offering a single GSoC coding period timeframe this year, starting on **Jun 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.*

Coding period	• I will adhere to the above deadlines.

Communication Channels

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	• Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I would prefer to communicate via email and Google Chat. Also, I can discuss the progress, issues, and workarounds with my mentors, via Google Meet, around two or three times per week.

Contributor Briefing

Note: *A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.*

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 10.	• Yes

Section 2: About Your Project

Project Details

Project title *(should match the one on the Project Ideas list)*	Lesson player redesign
Project size *(should match the*	Large (~350 hours)

options on the Project Ideas list)	
Why did you choose this project?	I chose this project because it aligns with my understanding and interests, and I'm excited to work on it. After exploring the codebase, I feel confident in handling the technical aspects, given my experience with similar challenges. I'm ready to take responsibility and am sure I can deliver it within the deadline.

Required Skills

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc.	*E2E test debugging topic editor* *Hint Editor Character Count Logic #18505*
I can debug and fix CI failures / flakes.	*E2E test debugging topic editor*
I can make changes to GitHub Actions workflows.	*#20683* *#21533*
WEB	
I can write Python code with unit tests.	*#20248* *#21142* *#21309*
I can write TS + Angular code with unit tests.	*#20248* *#19518* *#21823*
I can write or modify acceptance tests.	*#20662*
I can figure out reproduction steps based on information given in the server logs.	*#19111*

(Optional) Oppia Community References

Who you worked with	**Context in which you worked with them**
Aanuoluwapo Adeoti	Had many discussions on the project ideas as well as bridging the gap between the lesson creation team and the technical team.
Mohit Ruwatia	Worked on many angular migration issues and managed the LaCe team
Sambhav Kaushik	Worked in the cross-collaboration for the CD team and the LaCe team

Section 3: Proposal Details

Problem Statement

Oppia currently uses the Exploration Player to deliver community library lessons, curated lessons (classrooms), diagnostic tests, and practice questions. However, feedback received from Oppia learners revealed that the current design is confusing and inefficient for effective learning in the following ways:

1. **Lesson Player Issues**:
    - **Hints & Solutions**: Hard to find and not clearly differentiated. Users struggle to determine whether they are viewing a hint or a solution, which can cause confusion and hinder learning.
    - **Audio Voiceover**: The player is not easily discoverable on desktop, while on mobile, it takes up excessive space, reducing visibility for lesson content.
    - **Progress Tracking**: Learners find it difficult to track their lesson completion as progress indicators are hidden and not intuitive.
    - **Navigation Challenges**: There is no clear way for learners to revisit previous lessons or transition smoothly to the next lesson, making the lesson flow less seamless.
    - **Feedback & Responses**: Learners can view past responses, but the interface does not make it clear how previous mistakes should inform their current learning.
2. **Mobile Limitations**:
    - The lesson content area is significantly reduced due to the large audio player, headers, and hint bars.
    - The layout makes it challenging for learners to focus on lesson interactions while using mobile devices.
    - Limited space leads to a cluttered UI, making it harder to navigate lesson elements efficiently.

This project aims to redesign the Exploration Player to enhance clarity, usability, and learning efficiency while ensuring seamless functionality across all components, including community lessons, curated lessons, diagnostic test player, practice question player and the lesson player in the iframe mode.

Target Audience	- Learners
Core User Need	- As a learner, I want the lesson player to be more interactive and engaging. - As a learner, I want to easily access hints and solutions on both desktop and mobile.

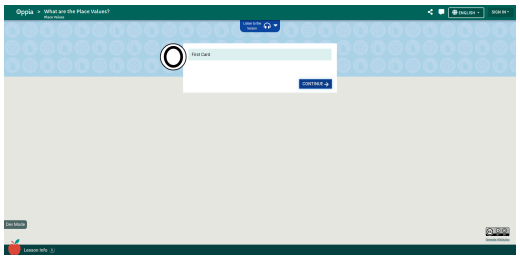
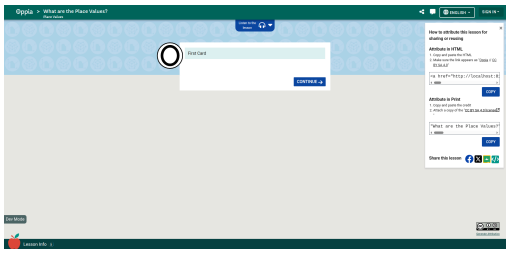
Section 3.1: WHAT

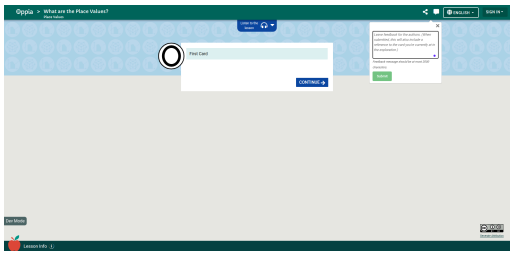
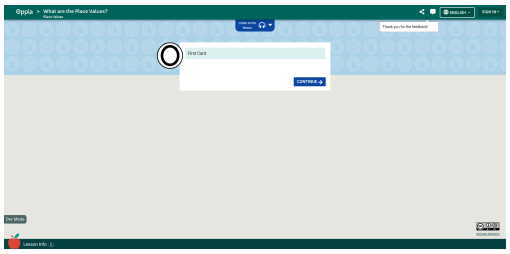
Key User Stories and Tasks that will be implemented & Testing Plan

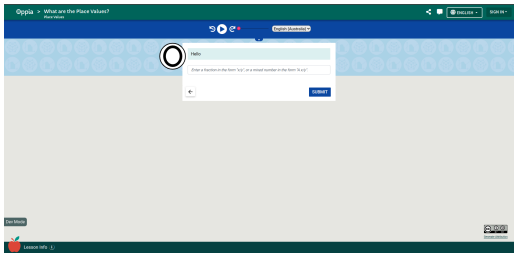
Critical user journeys

1. Learner (logged-out) playing an exploration (EL)

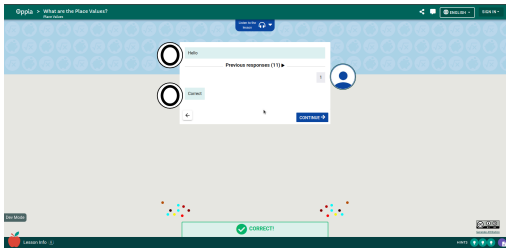
Goal	Steps	Expectations
Lesson Player CUJs		
Setup for the lesson player CUJs	1. Log in as a Curriculum Admin with the username "CurriculumAdmin". 2. Navigate to the exploration editor. 3. Create an exploration with the title "What are the Place Values?". 4. Select the Continue interaction for the first card. 5. Select the fraction interaction for the second card. (Just for the fraction interaction, add 2 hints and a solution.) 7. Now, click on the translations tab, and add the voiceover for the fraction interaction card for all, i.e., content, interaction, hint, and solution, in 2 languages: English and Hindi. 6. Now, add all the remaining interactions like “Multiple choice”, “Item selection”, “Image Region”, etc. 8. Select the End Exploration interaction for the last. 9. Set 1st, 2nd and 3rd card as checkpoints. 10. Save and publish. 11. Generate 2 dummy explorations with the titles "Explore Title 1" and "Explore Title 2". 12. Navigate to the exploration editor. Add 2 story nodes to each exploration. Select the Continue interaction for the first and the End Exploration interaction for the last. Save and publish. 13. Create a skill with the name “skil-1” and 10 questions inside it. 14. Create a Math classroom with a "Place Values" topic, - The "Place Values" topic should have 1 story and in that 3 chapters with the following titles: "What are the Place Values?", "Explore Title 1" and "Explore Title 2". 15. Create a subtopic for the "Place Values" topic and link "skill"_1 with it. 16. Enable the practice tab for it and publish the changes. 17. Navigate to the creator dashboard and choose the “What are the Place Values?” 18. Link the concept card with the first card of the lesson. 19. Save and publish. 20. Generate an exploration with the title "Community Lesson Title 1". 21. Navigate to the exploration editor. Add 2 story nodes to the exploration. Select the Continue interaction for the first (for the question content, use all the RTE components) and End Exploration interaction for the last. Save and publish. 22. Generate 2 dummy explorations with the titles "Community Lesson Title 2" and "Community Lesson Title 3". 23. Navigate to the exploration editor. Add 2 story nodes to each exploration. Select the Continue interaction for the first and the End Exploration interaction for the last. Save and publish.	

	24. Log out.		
EL.LP. Learner can access the lesson player			1
1.1	Failed to access the lesson player for a nonexistent lesson	Type /lesson/<wrong_lesson_id> into the URL bar.	- User lands on the 404 error page
1.2	Access the lesson player on the /lesson route	Type /lesson/<lesson_id> into the URL bar.	- User lands on the lesson player -The sidebar is in a collapsed state, and you can see the "Open Options" text in the sidebar - The page should look like this at this point: 
EL.SL. Learner can share the lesson from the lesson player			2
1.1	Share the lesson by copying the link	- Visit any lesson - Click on the “Open Options” button in the sidebar	The sidebar is in the expanded state, and the following things can be seen: - "Close Options" text in the sidebar - Lesson Description - Lesson Rating - Share button - Feedback Button - Report Button is not displayed
		- Click on the “Share” button in the sidebar - Click on the “Copy link” button.	“Link Copied” text appears in the share modal - The page should look like this at this point: 
		- Paste the copied link in a new tab - Press the Enter key	The lesson name is visible in the lesson player’s header
1.2	Share the lesson on Google Classroom	Visit any lesson - Click on the “Share” button in the sidebar - Click on the “Google	A new window pops up with “classroom.google.com” in the URL

		Classroom” logo	
1.3	Share the lesson on Facebook	Visit any lesson - Click on the “Share” button in the sidebar - Click on the “Facebook” logo	A new window pops up with “facebook.com” in the URL
1.4	Claim the lesson attribution	- Visit any lesson - Click on the “Share” button in the sidebar - Click on “How to attribute this lesson?”	“Generate Creative Commons Attribution” text is visible in the modal header
		Click on the “Copy Attribution” button in the attribution modal.	“Attribution copied” text is visible
		Paste the copied link in the text file	The link content is present in the file
EL.FB. Learner can give feedback on the lesson from the lesson player			3
1.1	Give feedback from the sidebar	- Visit any lesson - Click on the “Feedback” button in the sidebar	- The feedback modal pops up, and the “Give feedback on this lesson” text is visible in the modal header - “Stay anonymous” text is not visible - The page should look like this at this point: 
		- Write down the feedback in the input box shown in the modal. - Click on the “Submit” button	The thank-you modal shows up, with “Thank you for the feedback” in the modal header. - The page should look like this at this point: 
EL.VO. Learner can listen to voiceovers of the lessons in the lesson player			4
1.1	Play/Pause the audio	- Visit the Math classroom page	"Listen to this lesson in" text is visible

		- Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list - Click on the continue button in the footer	- Audio bar is visible - English is selected in the dropdown - Play/Pause is in paused state
		Click on the Play button	- English audio is played - The page should look like this at this point: 
1.2	Change the lesson audio language	- Click on the lesson audio dropdown - Select the Hindi language - Click on the play button	Hindi audio is played
1.3	Skip some parts of the lesson audio	Drag the audio bar slider forward	The audio starts playing from the point where the slider was released
1.4	Play the audio till the end	Click on the Play button	- The slider has moved to the start of the audio bar - Audio is in the paused state - The play button icon is displayed
EL.HC. Learner can see the feedback and help cards			5
1.1	See the first card of the lesson	- Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- The first card of the exploration is displayed. - The checkpoint bar is focused on the first node, and no node is colored. - The continue button is displayed in the footer. - “Save” button is displayed in the footer
1.2	Check the concept card	Wait for a few minutes	“View concept card” text is visible in the conversation display.
		Click on the “Concept card” button	Concept card modal pops up, and “Concept card” text is visible in the modal header
1.3	Go forward by one card	- Click on the close button - Click on the continue button in the footer	- Fraction interaction is displayed. - A back button that is enabled and a continue button that is disabled are displayed in the footer.

			- Checkpoint reached, celebration component appears - The checkpoint bar's first node gets colored, and now the 2nd node has the focus. - An input box and a submit button are also displayed above the footer.
1.4	Go backward by one card	Click on the back button	- The first card of exploration is displayed. - The checkpoint bar's first node ~~gets~~ is colored, and ~~now the 2nd node has the focus~~ the 2nd node remains focused. - The continue button is displayed in the footer.
1.5	Get feedback on the incorrect answer	- Click on the continue button in the footer - Enter the wrong answer in the input box - Click on the Submit button	- Answer is submitted - Feedback is received for the wrong answer
1.6	Get a hint or a solution when the user gets stuck	- Enter any text in the input box, e.g. "ABC" - Click on the submit button	- Error text is displayed below the input box. - The input box value is cleared - The submit button has been disabled
		Wait for a few minutes	“View hint” text is visible in the conversation display
		Click on the "View Hint" button	Hint modal pops up, and "Hint” text is visible in the modal header
		- Click on the "X" icon in the hint modal - Wait for a few minutes	“View hint 2” text is visible in the conversation display
		Click on the "View Hint 2" button	Hint modal pops up, and "Hint 2” text is visible in the modal header
		- Click on the "X" icon in the hint_2 modal - Wait for a few minutes	“View solution” text is visible in the conversation display
		Click on the "View Solution" button	- A modal pops up, and "Warning” text is visible in the modal header - “Show Solution” button is also visible
		Click on the “Show Solution” button	Solution modal pops up, and "Solution” text is visible in the modal header
1.7	Submit the correct answer and see the	- Enter the correct answer in the input box	- The submit button vanishes, and the Continue button shows up

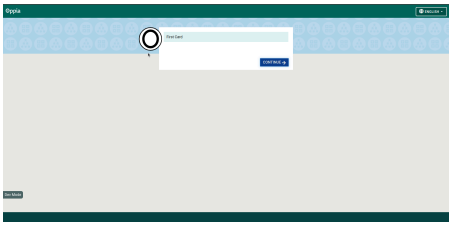
	celebration pop-up	- Click on the Submit button	- Celebration pop-up shows up - The page should look like this at this point: 
EL.CP. Learner reaches a checkpoint and saves their progress			6
1.1	Resume progress using the 72-hour link	Click on the “Save” button in the footer	The save-progress modal is displayed, and the “Save Progress” text is played in the modal header.
		- Click on the “copy” button - Copy the link to a new tab	The progress-reminder modal is displayed, with “Do you want to continue” displayed in the modal body.
		Click on the “Yes, resume the lesson” button	- The fraction interaction card is displayed - Sign-in button is displayed in the navbar
1.2	Sign up to permanently save the progress	- Close the new tab - Click on the “Create an Account” button	The login page is displayed
		Sign up as a new user	The progress-reminder modal is displayed, with “Do you want to continue” displayed in the modal body.
		Click on the “Yes, resume the lesson” button	- There is no “Save” button in the footer - A circular-shaped profile user avatar can be seen in the navbar at the top right - Sign-in button is not displayed
EL.CE. Learner completes the exploration and decides what to do next			7
1.1	Visit the next lesson	- Click on the profile menu and log out.	- It redirects to Oppia.org, and the home page is shown
		- Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- There is a “Save” button in the footer - A circular-shaped profile user avatar cannot be seen in the navbar at the top right - Sign-in button is displayed
		- Now, play through all the cards successfully and complete the exploration.	- “Sign up or Login” button appears - The next lesson button appears

			- The Check Review Card button appears - The " Do more practice button appears
		- Enter the feedback in the input box - Click on the submit button	The feedback modal is closed
		Right-click on the “Next Lesson” button to open it in a new tab.	“Explore Title 1” text is displayed in the lesson player’s header
1.2	Visit the practice tab	- Close the newly opened tab. - Click on the “Do more practice” button and open it in a new tab.	The practice tab of the “Place Values” topic is displayed in the new tab
1.3	Check the review card	- Close the newly opened tab. - Click on the “Check Review Card” button and open it in a new tab.	The revision tab of the “Place Values” topic is displayed in the new tab
1.4	Sign up or Login	- Close the newly opened tab. - Click on the “Sign up or Login” button and open it in a new tab.	The signup page is displayed in a new tab
EL.CL. Learner can play a complete community lesson			8
1.1	Use all the RTE components in a community lesson	Visit the community library	- "Community Lesson Title 1", "Community Lesson Title 2" and "Community Lesson Title 3"; these lessons are displayed
		Click on the "Community Lesson Title 1"	- User lands on the lesson player -The sidebar is in a collapsed state, and you can see the "Open Options" text in the sidebar
		Click on the continue button in the footer	Fraction interaction is displayed. - A back button that is enabled and a continue button that is disabled is displayed in the footer.
		Click on the concept card RTE component	Concept card modal is displayed, and “Concept card” text is visible in the modal header
		- Click on the close button - Click on the video RTE component play button	- Video starts playing
		- Click on the link RTE component	- Link is opened in a new tab
		- Close the new tab	- Collapsible RTE component content is

		- Click on the collapsible RTE.	displayed
		- Click on the tab RTE component	- Tab RTE component content is switched with the new tab content
EL.NL. Learner completes the community exploration and decides what to do next			9
1.1	Visit the community library	Play the lesson till the end card	- The community library button is visible - The next button is visible - Rating stars are not visible –"Community Lesson Title 2" and "Community Lesson Title 3" text is also visible - Sign-in button should be visible
		Click on the “Go to library” button and open it in a new tab.	The community library page is displayed
1.2	Visit the next community lesson	- Close the newly opened tab. - Click on the "Community Lesson Title 2" button and open it in a new tab.	“Community lesson: Community Lesson Title 2” text is visible in the lesson player header.

2. Learner (logged-out) plays an embedded exploration

Goal		Steps	Expectations	
EE. User can embed the lesson on their website				1
1.1	Embed Oppia lessons	- Visit any lesson - Click on the “Share” button in the sidebar - Share modal pops up - Click on the “Embed in Webpage” button in the share modal.	“Embed in Webpage” text is visible in the modal header	
		Click on the “Embed in Webpage” button	“HTML code copied” text is visible	
		Paste the copied link in a file	The link content is present in the file	
EE. Learner can complete the embedded lesson				2
1.1	Starting an embedded lesson	- Visit /embed/exploration/<exp_id>	- The first card of exploration is displayed. - The continue button is displayed in the footer. - The checkpoint bar is not visible in the	

			footer - The audio bar is not visible - In the site navbar, the language change option should be visible, and the default language set as “English” - The page should look like this at this point: 
1.2	Completing an embedded lesson	- Visit /embed/exploration/<exp_id> - Complete the lesson to the end	- Lesson completion confetti shows up - No chapter suggestion is shown

3. Learner (logged-in) (LI)

Goal	Steps	Expectations
Lesson Player CUJs		
Set up for the lesson player CUJs	1. Log in as a Curriculum Admin with the username "CurriculumAdmin". 2. Navigate to the exploration editor. 3. Create an exploration with the title "What are the Place Values?". 4. Select the Continue interaction for the first card. 5. Select the fraction interaction for the second card. 6. Select the Continue interaction for the third and fourth card. 7. Select the End Exploration interaction for the last. 9. Set 1st, 2nd, 4th and last card as checkpoints. 10. Save and publish. 11. Generate 2 dummy explorations with the titles "Explore Title 1" and "Explore Title 2". 12. Navigate to the exploration editor. Add 2 story nodes to each exploration. Select the Continue interaction for the first and the End Exploration interaction for the last. Save and publish. 13. Create a Math classroom with a "Place Values" topic. The "Place Values" topic should have 1 story and in that 3 chapters with the following titles: "What are the Place Values?", "Explore Title 1" and "Explore Title 2". 14. Navigate to the creator dashboard and choose the “What are the Place Values?” 15. Save and publish. 16. Log out. 17. Log in as a moderator with the username "Moderator". 18. Log out 19. Log in as a user with username "Learner" and email user@example.com	

LI.OP Learner cannot access pages that require higher privileges				1
1.1	Failed to access the contributor dashboard admin page	Type “/contributor-admin-dashboard” into the URL bar.	User lands on the 401 error page	
1.2	Failed to access the topics and skills dashboard page	Type “/topics-and-skills-dashboard” into the URL bar.	User lands on the 401 error page	
1.3	Failed to access the release coordinator page	Type “/release-coordinator” into the URL bar.	User lands on the 401 error page	
1.4	Failed to access the moderator page	Type “/moderator” into the URL bar.	User lands on the 401 error page	
1.5	Failed to access the site admin page	Type “/admin” into the URL bar.	User lands on the 401 error page	
LI. Learner can access the lesson player				2
1.1	Failed to access the lesson player for a nonexistent lesson	Type /lesson/<wrong_lesson_id> into the URL bar.	- User lands on the 404 error page	
1.2	Access the lesson player on the /lesson route	Type /lesson/<lesson_id> into the URL bar.	- User lands on the lesson player -The sidebar is in a collapsed state, and you can see the "Open Options" text in the sidebar	
LI. Learner can provide feedback on the lesson or report it from the lesson player				3
1.1	Report the lesson from the sidebar	- Visit any lesson - Click on the “Report” button in the sidebar	The “Report this lesson” modal pops up, and the "Report this lesson" text is visible in the modal header	
		- Select any one reason from it - Write down the detailed reason in the input box shown in the modal. - Click on the “Submit” button	“Your report has been forwarded to the moderators for review.” text is visible in the modal body	
1.2	Report sent to moderator [Not implementable in acceptance tests], but this journey should be validated using manual testing.	Check the moderator's mail	An email has been received with the report.	
LI.SP Learner reaches a checkpoint and saves their progress				4

1.1	Track the Checkpoint progress	- Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- The first card of the exploration is displayed. - The checkpoint bar is focused on the first node, and no node is colored. - The continue button is displayed in the footer.
		- Click on the continue button in the footer	- Fraction interaction is displayed. - A back button that is enabled and a continue button that is disabled are displayed in the footer. - Checkpoint reached, celebration component appears - The checkpoint bar's first node gets colored, and now the 2nd node has the focus. - An input box and a submit button are also displayed above the footer.
		- Enter the correct answer in the input box - Click on the Submit button	- The submit button vanishes, and the Continue button shows up - Celebration pop-up shows up
		Click on the continue button in the footer	- New card appears - The checkpoint bar has 2 nodes with color-filled
1.2	Resume the lesson from the last progress saved	- Click on the profile menu in the navbar. - Click on the learner dashboard - Use the navbar to visit the math classroom - Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- The progress reminder modal is displayed, and the “Do you want to continue” text is displayed in the modal body. - The checkpoint bar shown in the modal body has 2 nodes with color-filled.
		Click on the "Yes, resume the lesson" button	3rd card is displayed
1.3	Restart the lesson from the beginning	- Click on the profile menu in the navbar. - Click on the learner dashboard - Use the navbar to visit the math classroom - Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- The first card of exploration is displayed. - The checkpoint bar has focused on the first node, and no node is colored. - The continue button is displayed in the footer.

		- Click on the “No, restart from the beginning” button	
1.4	Exit the lesson player	Click on the continue button in the footer	- A back button that is enabled and a continue button that is disabled is displayed in the footer.
		Click on the back button in the browser tab.	The save progress modal is displayed
		Click on the cancel button	The save progress modal disappears
		Click on the "X" icon in the lesson player header	The save progress modal is displayed
		Click on the cancel button	The save progress modal disappears
LI. Learner completes the exploration and decides what to do next			5
1.1	Rate the lesson	Now, play through all the cards successfully and complete the exploration.	- 5 blank stars appear to rate this lesson - “Go to the learner dashboard” button appears - The next lesson button appears - The Check Review Card button appears - The " Do more practice button appears
		Click on the 3rd star	- 3 stars get colored and 2 remain uncoloured - Feedback modal appears
		- Enter the feedback in the input box - Click on the submit button	The feedback modal is closed
1.2	Visit the learner dashboard	- Close the newly opened tab. - Click on the “Go to Learner Dashboard” button and open it in a new tab.	The learner dashboard is displayed in a new tab
LI. Learner starts from beginning after completing a lesson			6
1.1	Once an exploration is completed the learner will start the exploration from the beginning the next time they visit it.	- Close the newly opened tab. - Visit the Math classroom page - Select the "Place Values" topic - Select the "What are the Place Values?" lesson from the list	- The first card of the exploration is displayed. - The checkpoint bar is focused on the first node, and no node is colored. - The continue button is displayed in the footer.

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if relevant, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	N/A	N/A	N/A	N/A

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.	N/A	N/A	N/A

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.	N/A	N/A	N/A

UI Screens/Components

Accessibility & Semantic Design Guidelines (Reference)

The following considerations should be kept in mind during implementation, even if not explicitly listed in every task. These serve as general standards for the redesigned UI:

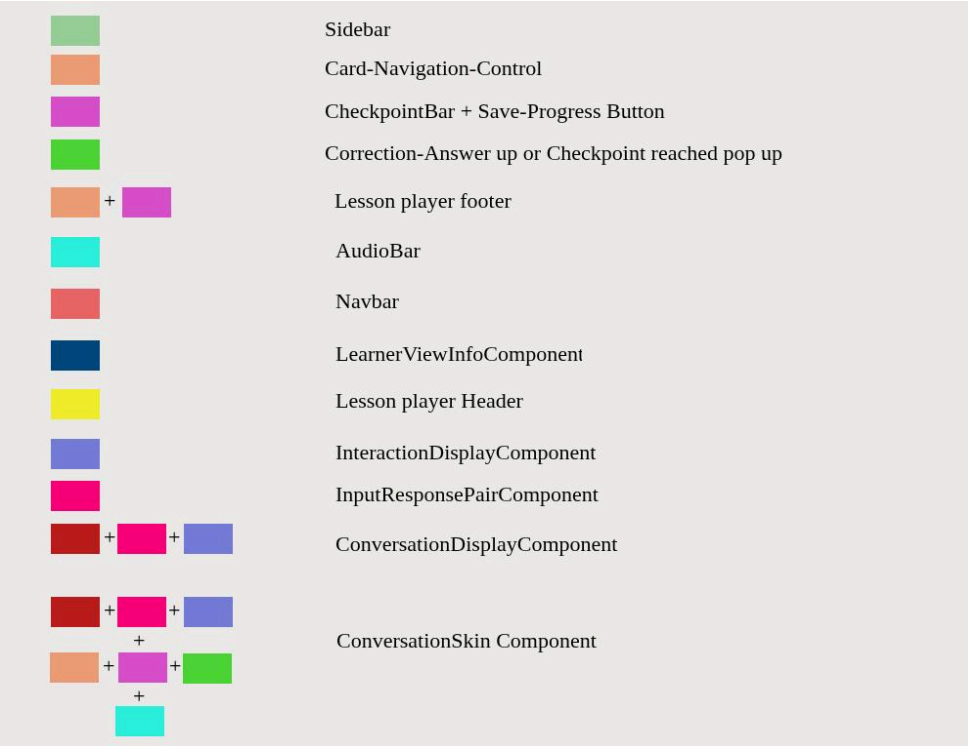
- **Inputs and labels** should be clearly associated for accessibility.
  Buttons with icons should include descriptive text for their purpose (note: the icon itself should not be described).
- Use **semantic HTML** where appropriate, including:
  - Proper use of headings (`<h1>`, `<h2>`, etc.)
  - Structured sections (`<section>`, `<article>`)
  - Meaningful links and buttons
- All **images** should include relevant `alt` text.
- **Links** should have descriptive anchor text (avoid "click here").

These practices help ensure usability, accessibility, and maintainability across the product.

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	Sidebar component	The sidebar will show the essential details of the lesson and will contain some helper buttons	Yes	Mock	Tabindex and focus state will be added so that it's accessible through tab navigation, and the aria-expanded attribute will also be added for screen-readers
2.	Feedback modal	The feedback modal will provide a way for learners to give feedback on lessons.	Yes	Mock	Same as above Use ARIA roles (`role="dialog"`) and labels.
3.	Share lesson modal	The share lesson modal will show the ways to share the lesson and will also show the embed link and attribution link.	Yes	Mock	Same as above
4.	AudioBar component	This helps the learner learn the lesson in the audio format.	Yes	Mock	Provide keyboard shortcuts for play, pause, and seek. Use ARIA live regions to announce playback status.
5.	Common Header component	This provides the title of the lesson player, diagnostic test, or practice session feature, and the way to exit these features.	Yes	Mock	Ensure the exit button is clearly labeled and can be accessed using the keyboard.
6.	Exit modal	This will appear as a warning to users when they try to exit the lesson because the progress will only be saved till the last checkpoint is complete.	Yes	Mock	Tabindex and focus state will be added so that it's accessible through tab navigation.
7.	New Skin component	The new skin will be the new design for the lesson player cards	Yes	Mock	Tabindex and focus state will be added so that hints and solutions can be accessed.

8.	Save Progress modal	The save progress modal will show up to logged-out users, to provide them a way to save their progress.	Yes	Mock	Tabindex and focus state will be added so that it's accessible through tab navigation.
9.	Checkpoint Bar	The checkpoint bar will show the learner's progress. We are creating a new component so that we can reuse the same component in the progress modal and the exit modal. The current implementation duplicates the same logic.	No	Mock	Ensure progress updates are announced using ARIA live regions.
10.	Common Progress Navigation component	This component will be used to forward or backward by one card, it will also be used to submit the answers for the non-text interactions. **Note:** The black part in the demo image will be used for the lesson player footer or practice session footer components.	Yes	Mock	Tabindex and focus state will be added so that it's accessible through tab navigation.
11.	Conversation display component	This component will be used to display the new design of the conversation display. For **Supplemental Interactions**: The tutor card is displayed alongside the supplemental card (canvas). The tutor card is responsible for presenting the question, showing the user's answer, and providing feedback. It does *not* handle input collection or submission for the supplemental card. The supplemental card (canvas) is responsible for rendering the interaction content and capturing user input through the conversation-skin component. Since the supplemental card canvas is not being redesigned, updating the tutor card alone will be sufficient to support the supplemental interaction experience (mocks). For **Inline Interactions**: We display	Yes	Mock	N/A

		the tutor-card along with the `oppia-interaction-display`. Similarly, since we are not redesigning the interactions themselves, the redesign of the tutor-card will be sufficient to handle inline interactions as well. (For more reference)			
12.	Lesson player footer component	This component will be used to display the checkpoint bar, save progress button, and lesson player modals. **Note:** This will be placed in the black part shown in the Common Progress Navigation component	Yes	Mock	Tabindex and focus state will be added so that it's accessible through tab navigation.



Data Handling and Privacy

It is not required, as no new user data is being collected.

Connection to Existing Work

Current Folder Structure:

```
exploration-player-page
├── layout-directives
├── learner-experience
├── modals
├── new-lesson-player
│   ├── new-lesson-player-components
│   └── new-lesson-player-services
├── services
└── templates
```

- The templates folder and modals folder both contain modals.
- The layout-directives folder contains the main components of the current lesson player, and some of these components can be reused in the new setup.
- The learner-experience folder contains conversation-skin related components that will be reused in the new lesson player.

Other Requirements

N/A

Questions Asked in the discussions

- https://github.com/oppia/oppia/discussions/22035
- https://github.com/oppia/oppia/discussions/22039
- https://github.com/oppia/oppia/discussions/22076
- https://github.com/oppia/oppia/discussions/22163
- https://github.com/oppia/oppia/discussions/22187
- https://github.com/oppia/oppia/discussions/22374

Section 3.2: HOW

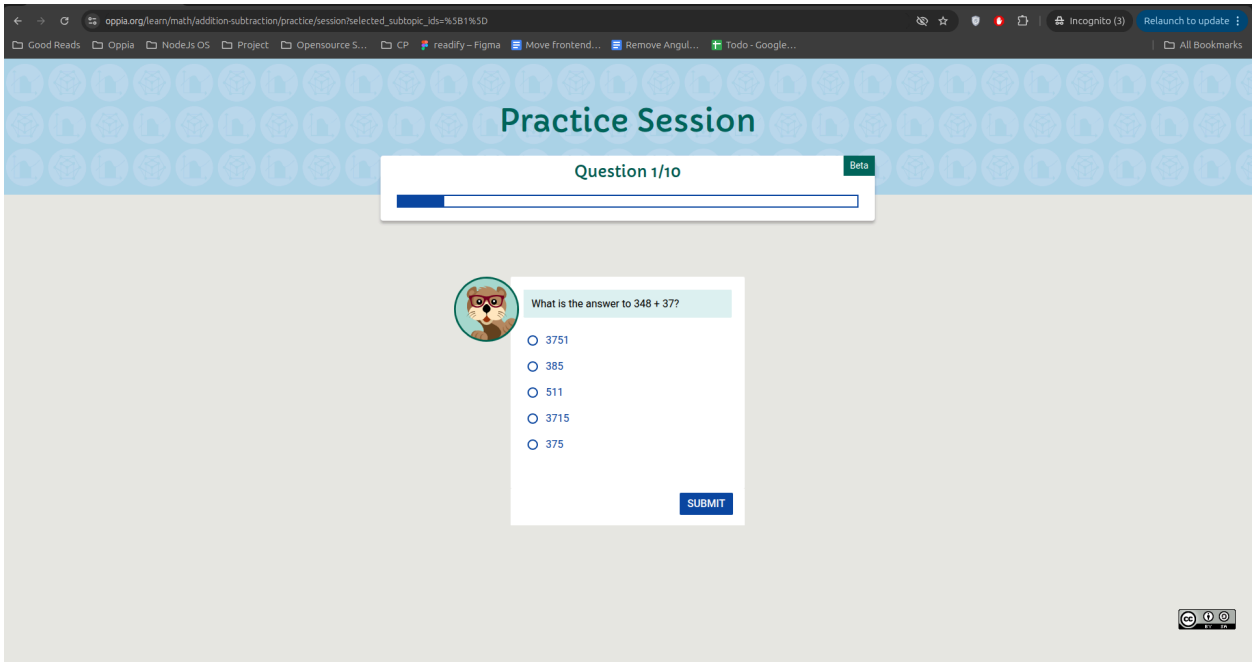
Existing Status Quo

1. Interaction connection with the conversation-skin component:-
   This block of code is responsible for rendering the inline interaction in the tutor card and tutor-card component is used in the conversation-skin to render the question, answers and feedback.

2. Supplemental interactions:-
   This block of code is responsible for rendering the supplemental card in desktop mode, while this other block of code handles the rendering of the supplemental card for small screens. Additionally, this block of code manages the question and feedback rendering functionality.

3. Iframe:-
   This feature is currently accessible through the "embed/exploration/:exploration_id" route. There is no need for any changes to the navbar in the existing design. However, the conversation skin and footer components will need to be updated (Image Reference).

4. Oppia-exploration-footer:-
   This component is currently used in three files: practice-session-page-root.component.html, exploration-player-page-root.component.html and preview-tab.component.html. We can safely remove it from the `practice-session-page` file, as it doesn't appear on that page, and we want to avoid displaying hints and solutions to learners to prevent them from rushing through the practice questions.



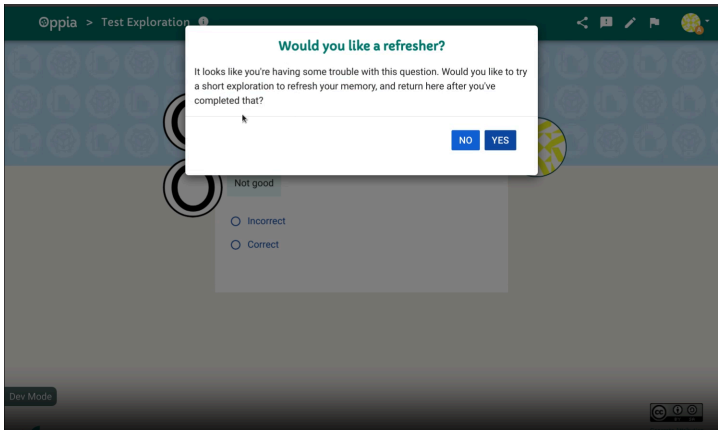
Features Pending Product Audit

The following features have been identified as potentially outdated, incomplete, or unused in the current codebase. However, before any removal or major changes can be made, a product audit is required to determine their future relevance. These are documented here for completeness and possible follow-up after audit decisions.

1. Refresher Exploration Confirmation Modal

- **Resource**: Pull Request #12748
- **Issue**: #22780
- **Status**: This feature is pending a product audit to assess its relevance and determine if it should be retained or removed.

- **GSoC Plan**: **Will not be addressed** in this project. It is being tracked for future cleanup.



2. Hints and Solutions in Question Player

- **Issue**: #22766
- **Status**: The feature exists in the codebase but is not currently exposed in the UI. A product audit is needed to determine its future.
- **GSoC Plan**: **Will be either removing or modifying code** as part of this project. Awaiting product team input.

3. Pretest Feature/Mode

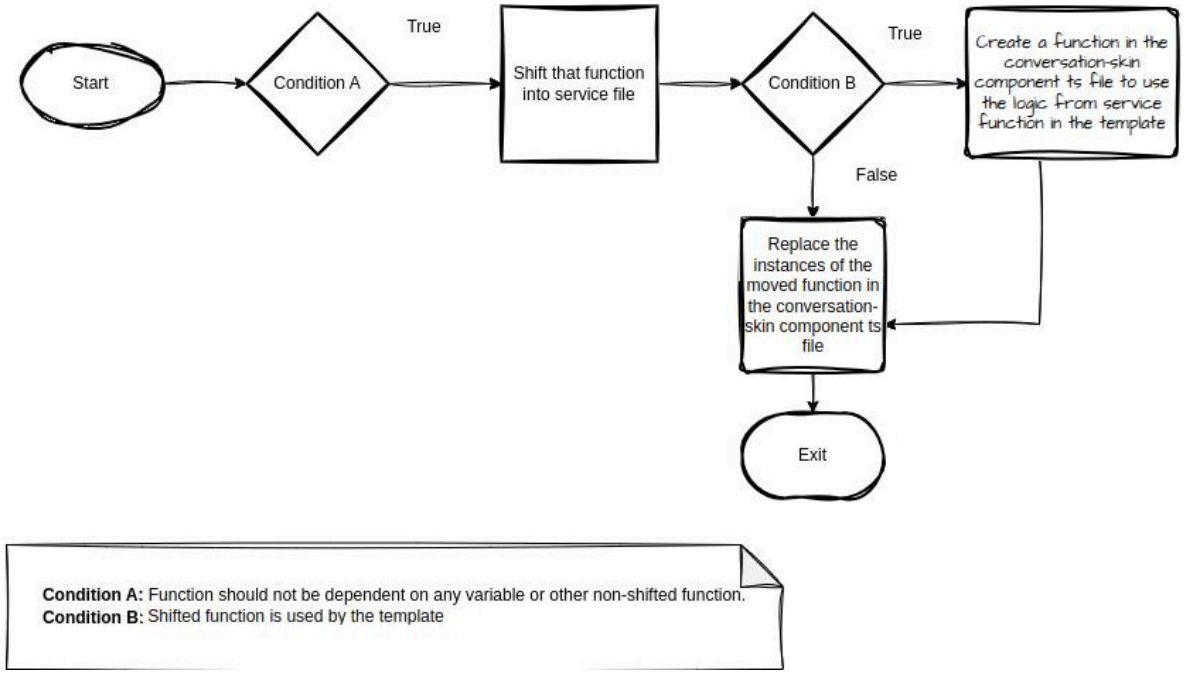
- **Resource**: Pull Request #5479
- **Issue**: #22770
- **Status**: This feature appears incomplete or non-functional. Pending a product decision on whether it should be developed or removed.
- **GSoC Plan**: **Will not be addressed** in this project. It is being tracked for future cleanup.

Solution Overview

1. Refactor conversation-skin.component.ts -

- There are two key steps to consider when moving the `conversation-skin` component logic into the service. Since there are many nested function dependencies and variables that are interconnected with several functions and templates, it becomes challenging to directly transfer the functions into the service file. To manage this, we can follow these steps:

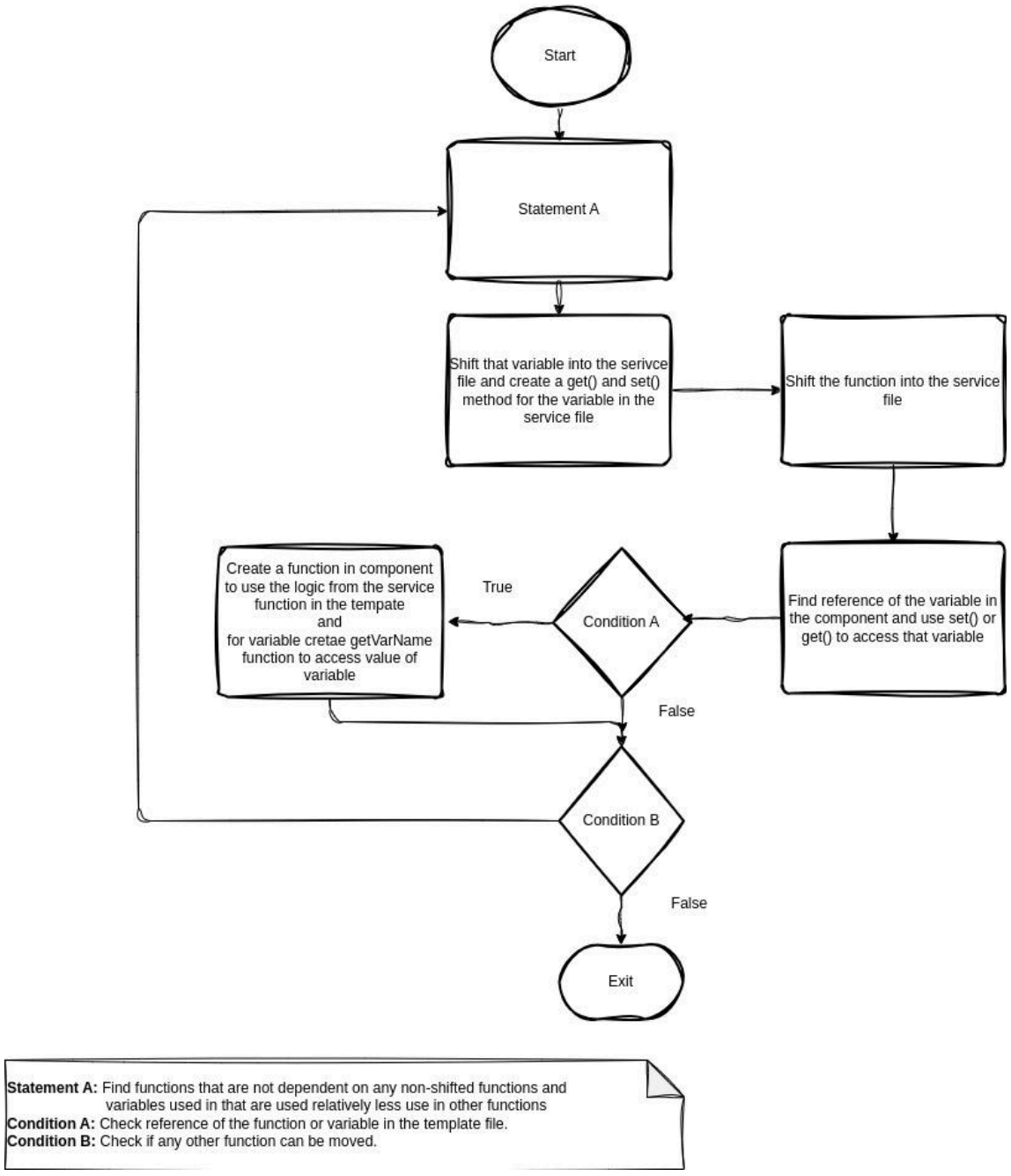
    - Moving the non-dependent functions and breaking complex functions into chunks.



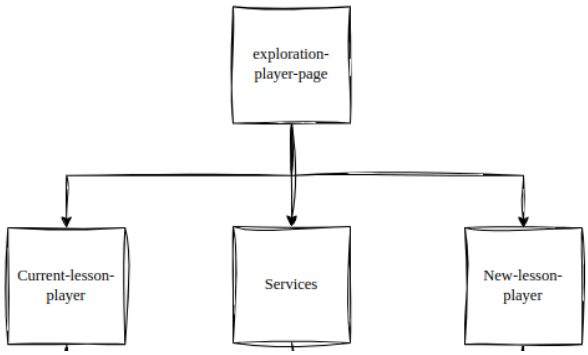
Example of a non-dependent function: _getRandomSuffix()

Example of breaking complex functions into chunks: The submitAnswer() function is currently around 200 lines long and performs multiple tasks within a single block of code, making it difficult to read and maintain. To improve clarity and maintainability, I have broken it down into some helper functions. This approach will helps us identify utility functions, which can then be moved into their respective services for better organization and reusability.

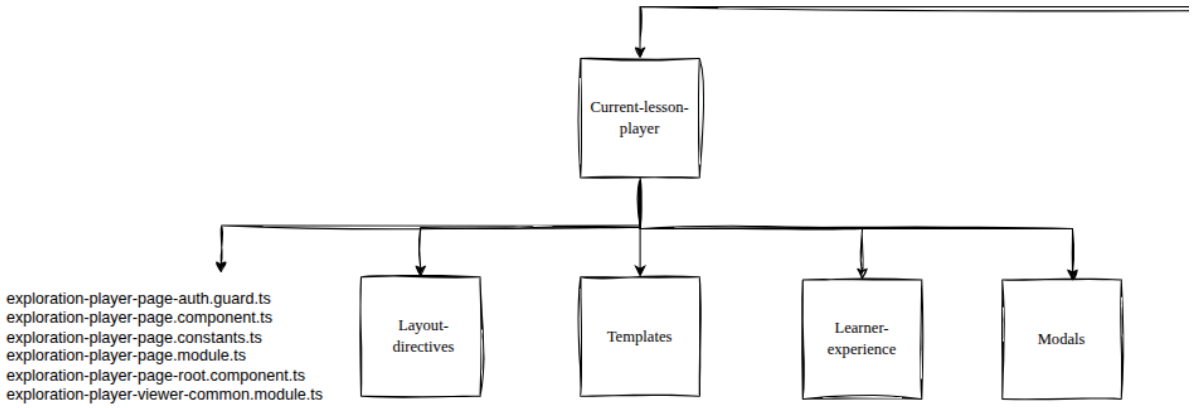
- Moving the dependent and complicated functions.



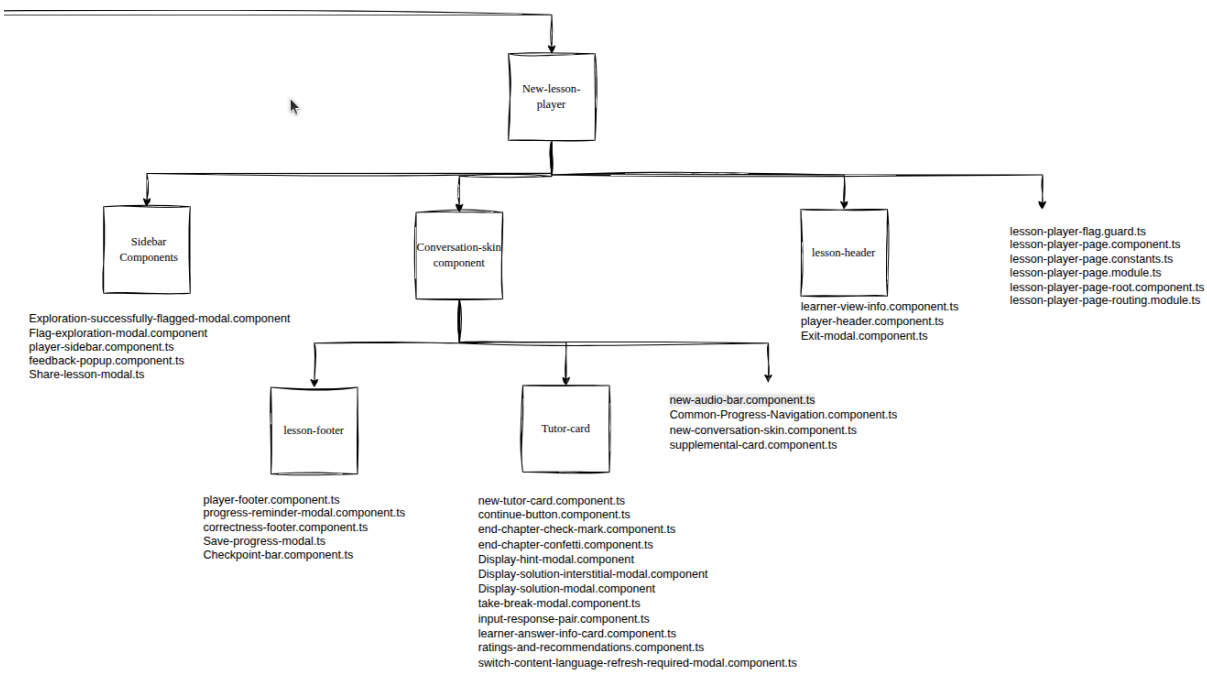
2. Implementation of the UI -
    - The current folder structure is mentioned in the Connection to Existing Work heading.
    - After refactoring the /exploration-player-page folder, we will have the 3 subfolders:



- The image below shows the structure of the current-lesson-player directory after moving the files to their respective folders. All the files shown in this directory will be removed once we launch the new-lesson-player.



- The image below shows the structure of the `new-lesson-player` directory. It contains some files and components that will be shared between both the `current-lesson-player` and `new-lesson-player`, as well as all the newly generated components specific to the `new-lesson-player`.



- **Sidebar Components Directory:** This directory will contain all the files and components related to the functionality of the sidebar in the lesson-player. Below is a brief description of each file:
    - **Exploration-successfully-flagged-modal.component.ts**: This is not a new file. It displays a "thanks" message modal for reporting a lesson.
    - **Flag-exploration-modal.component.ts**: This is not a new file. It handles the logic for the report modal.
    - **player-sidebar.component.ts**: This will be a new file, responsible for the logic of the lesson-player sidebar.
    - **feedback-popup.component.ts**: This is not a new file. It currently shows a pop-up asking for feedback. The logic will remain the same, but the template will be redesigned as a modal instead of a pop-up.
    - **share-lesson-modal.ts**: This will be a new file, responsible for managing the logic of the share modal.
- **Conversation-skin Components Directory:** This directory will contain all the files and components related to the functionality of the conversation flow in the lesson-player. The directory is divided into two subdirectories: **Lesson-footer Components** and **Tutor-card Components** (Will be renamed to Conversation Display Components). Below is a brief description of each file:
    - **Lesson-footer Components Directory:** This directory will contain all the files and components related to the functionality of the lesson footer (excluding progress-navigation) in the lesson-player.

        - **Progress-tracker.component.ts**: This component provides the UI for saving progress, interacting with related modals, and triggering checkpoint celebration events.
        - **progress-reminder-modal.component.ts**: This is not a new file. It is responsible for the logic of the progress-reminder moda.
        - **correctness-footer.component.ts**: This is not a new file. It shows the correct answer modal with a confetti.
        - **save-progress-modal.ts**: This will be a new file responsible for managing the logic of the save-progress modal.
        - **checkpoint-bar.component.ts**: This will be a new file used to manage the common checkpoint bar logic.
    - **Conversation Display Component Directory:** This directory will contain all the files and components related to the answer submission, question display, feedback on the answers, hints, solutions, and linked concepts cards.
        - **conversation-display.component.ts**: This is a new file. It will handle the display of the question, answer, feedback on the answer, solution, hints, linked concept card, and the answer submission logic.
        - **continue-button.component.ts**: This is not a new file. It maintains the logic for displaying the continue button.
        - **end-chapter-check-mark.component.ts**: This is not a new file. It is responsible for conditionally displaying the end-chapter check mark.
        - **end-chapter-confetti.component.ts**: This is not a new file. It handles the logic for displaying the end-chapter confetti.
        - **display-hint-modal.component.ts**: This is not a new file. It manages the logic for the display hint modal.
        - **display-solution-interstitial-modal.component.ts**: This is not a new file. It manages the logic for the display solution interstitial modal, which serves as a confirmation modal asking the user if they want to view the solution.
        - **display-solution-modal.component.ts**: This is not a new file. It handles the logic for the display solution modal.
        - **take-break-modal.component.ts**: This is not a new file. It manages the logic for the take-break modal, which is displayed when a user starts spamming answers.
        - **input-response-pair.component.ts**: It will handle the logic for input and response pairing.

        - **learner-answer-info-card.component.ts**: This is not a new file. It manages the logic for the learner answer info card, which asks for detailed answers. This functionality may not work properly or is rarely used.

        - **ratings-and-recommendations.component.ts**: This is not a new file. It manages the logic for lesson-end recommendations.

        - **switch-content-language-refresh-required-modal.component.ts**: This is not a new file. It manages the logic for the switch content modal.

    - **At the Root of the Conversation-skin Components Directory**: There will be three new files:
        - **new-audio-bar.component.ts**: This is a new file that will be used to display the new audio bar in the new conversation skin.

        - **card-navigation-controls.component.ts**: This is a new file that will display the progress navigation bar at the bottom of the new conversation skin.

        - **new-conversation-skin.component.ts**: This is a new file that will affect the diagnostic test player, practice question player, and the new lesson player. It will combine the audio bar, progress navigation, lesson footer (In lesson player), and tutor card component.

        - **supplemental-card.component.ts**: This is not a new file. It manages the logic for displaying the supplemental card.

- **Lesson-header Components Directory:** This directory will contain all the files and components related to the functionality of the header in the lesson-player. Below is a brief description of each file:
    - **navbar-breadcrumb.component.ts** : This is not a new file. It displays the breadcrumb in the navbar for the lesson player (Previously known as learner-view-info.component.ts).

    - **player-header.component.ts**: This is a new file that will be used for displaying the lesson header and lesson exit functionality. It will be used in the practice-question player, diagnostic test player, and the lesson-player.

    - **exit-modal.component.ts**: This is a new file that will manage the logic for the exit modal.

- **At the Root of the new-lesson-player Directory:**
    - **lesson-player-flag.guard.ts**: This is a new file used to gate the `/lesson/<lesson_id>` route using a frontend feature flag. This introduces a **new pattern** for route-level feature gating on the frontend, which differs from existing projects that typically perform gating in the backend via `access_validators.py`.

      Once the flag is removed, this file can be renamed to `lesson-player-auth.guard.ts`, and the logic should be updated to align with how

route guards are typically handled (e.g., consistent with `auth.guard.ts`).

- **lesson-player-page.component.ts**: This is a new file that will combine the necessary components to complete the new lesson player. It will include the Common-header component, Sidebar component, and new-conversation component to create the new lesson player.
- **lesson-player-page.constants.ts**: This is a new file that will contain all the constants used within the new lesson player.
- **lesson-player-page.module.ts**: This is a new file that will contain all the references to services and components needed for the new lesson player.
- **lesson-player-page-root.component.ts**: This is a new file that will combine the new lesson player components, base components, and navbar to form the root component for the new lesson player.
- **lesson-player-page-routing.module.ts**: This is a new file to handle routing for the new lesson player. Later on, we can merge this with the `auth guard` file.

- Each sub-directory will also include a `README.md` file at its root, providing detailed information about the files and folders contained within that directory to avoid any confusion. The following README.md files will be created for each folder:

/exploration-player/README.md

This directory contains the components and services related to both the current and new lesson player implementations. It includes the following sub-directories:

- `/current-lesson-player`: Contains files and components used in the existing lesson player. This will be deprecated after the full launch of the new lesson player.
- `/new-lesson-player`: Contains all components necessary for the new lesson player experience.
- `/services`: Contains shared logic and service files used by both the current and new lesson players.

/exploration-player/current-lesson-player/README.md

⚠️ **Deprecated Folder Warning**

This folder contains all files and sub-directories that will be removed once the new lesson player is fully launched.

What is the "New Lesson Player"?

The **new lesson player** is a redesigned and modernized version of the interface learners use to go through lessons (explorations). It aims to improve:

- **User experience and accessibility**
- **Responsiveness on mobile devices**
- **Code maintainability and modularity**

It likely involves a rearchitecture of the frontend components and services that power the lesson-viewing experience.

How to Check Its Current Status

You can follow the development status and rollout of the new lesson player through **GitHub issue #19217**. That issue contains:

- A full **milestone table**
- **Target dates** for key feature completions
- Details on what functionality is already complete

Guidelines:

- **Avoid adding new files.**
  If it is absolutely necessary to add or modify files in this folder, please **contact Hardik** at hardikgoyal2003@gmail.com before proceeding.

Sub-directories:

- `/layout-directives`: Layout-related directives used by the current player.
- `/learner-experience`: Learner-focused components in the current player.
- `/modals`: Modal components used within the current player.

/exploration-player/new-lesson-player/README.md

This folder contains all necessary components for the proper functioning of the **New Lesson Player**.

What is the "New Lesson Player"?

The **new lesson player** is a redesigned and modernized version of the interface learners use to go through lessons (explorations). It aims to improve:

- **User experience and accessibility**
- **Responsiveness on mobile devices**
- **Code maintainability and modularity**

It likely involves a rearchitecture of the frontend components and services that power the lesson-viewing experience.

How to Check Its Current Status

You can follow the development status and rollout of the new lesson player through **GitHub issue #19217**. That issue contains:

- A full **milestone table**
- **Target dates** for key feature completions
- Details on what functionality is already complete

Sub-directories:

- `/sidebar-component`: Contains all files required for the sidebar UI and its functionalities.
- `/conversation-skin-components`: Includes components related to the conversation skin, like supplemental card, input-response, hints, concept card, solution, etc.
    - `/Progress-tracker`: Manages checkpoint functionality, progress-saving, and celebration pop-ups.
    - `/conversation-display`: This folder contains all the functionalities related to the display and behavior within the conversation interface.
- `/lesson-header`: This folder contains all the components and logic required to display and manage the lesson header information.

/exploration-player/Services/README.md

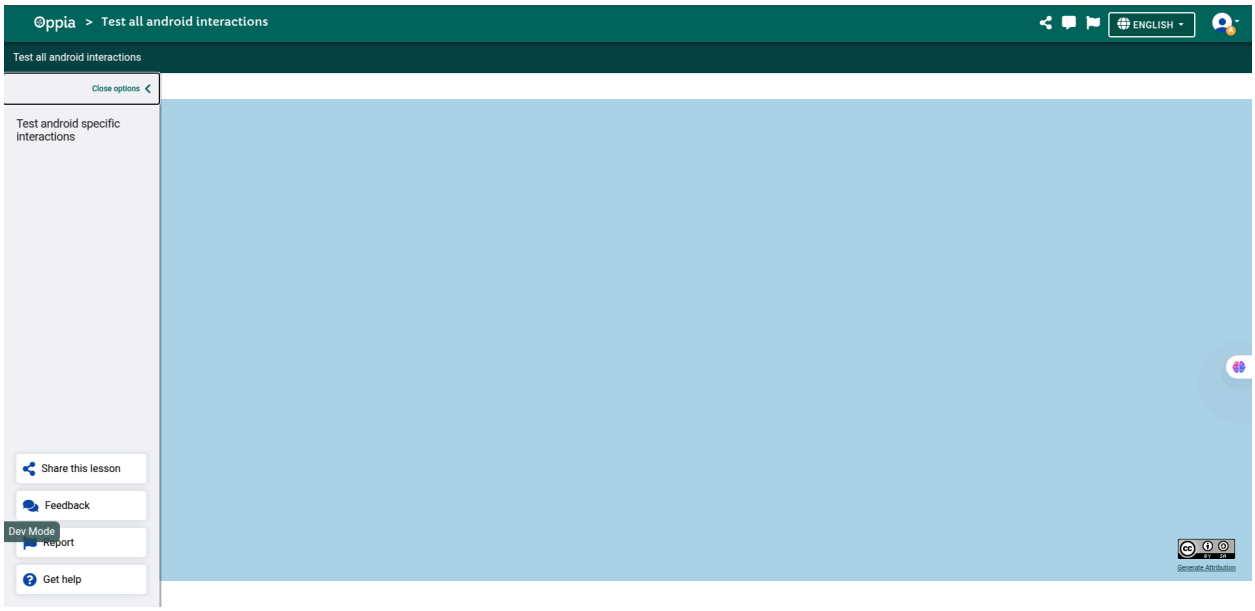
This folder contains all the **shared service files** essential for the functionality of the lesson players.

Guidelines:

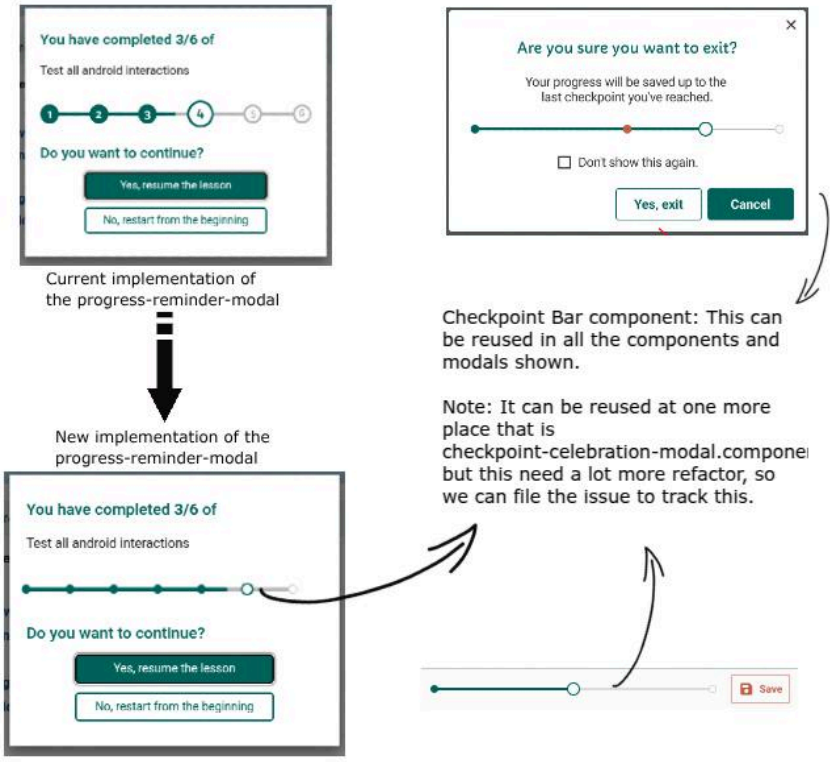
- Whenever you **add a new service** or **modify existing methods**, ensure that proper **JsDoc** is written for them.
- These services are **shared across multiple components and pages**, so changes should be made with caution and proper testing. For the proper testing, below are the recommendations that should be followed:

- **Writing or updating unit tests** for the service itself to verify all expected behaviors, edge cases, and error handling.
- **Running tests for all dependent components** to ensure nothing breaks due to the service change.
- Navigate through the app and **manually verify the behavior** of features that **rely on the service**.

- Lesson Player initial structure: I will use the layout introduced in this PR for the new lesson player.



- Reusable checkpoint bar Component



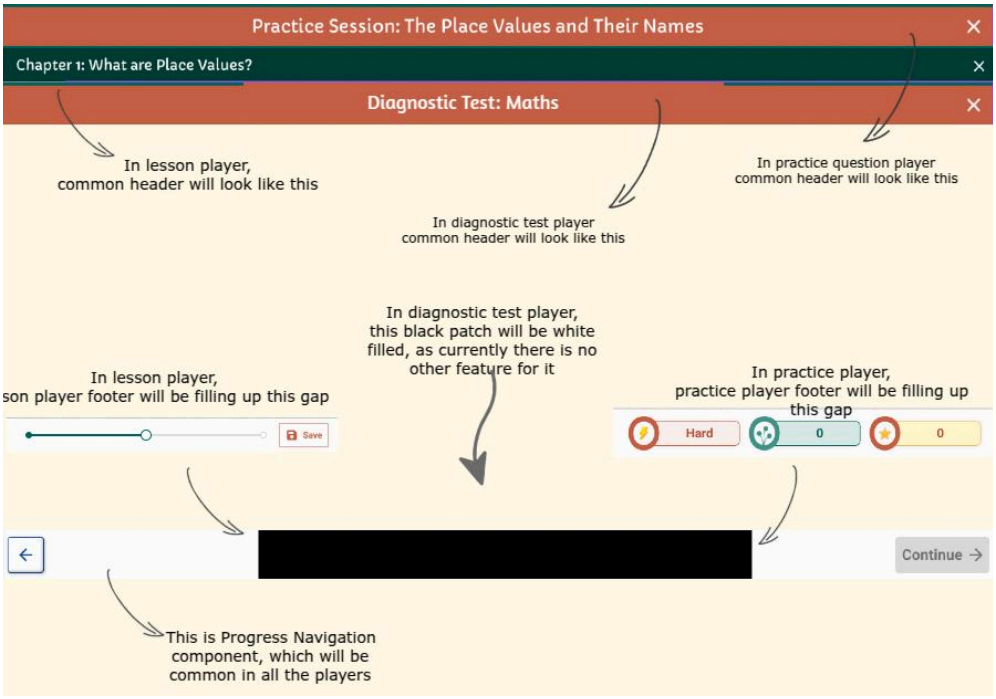
To avoid skew with the current implementation of the progress-reminder modal:

- In progress-reminder-modal.component.html from line 11-19 will be gated by *ngIf, when the feature flag is off, this will be rendered.
- When feature flag will be turned on, then <oppia-checkpoint-bar> will be rendered.
- <oppia-checkpoint-bar [checkpointBarShowsNumber] = “‘true”’>, it will be used where when we need to render the checkpointBar with the numbers, else by default it will render the non-numbered checkpointBar.

- Reusable header / progress navigation component

The following diagram and table are included to **document the new product experience**. They outline how users will interact with the system and help clarify the new flows and functionality.

This is important for ensuring product behavior aligns with expectations.

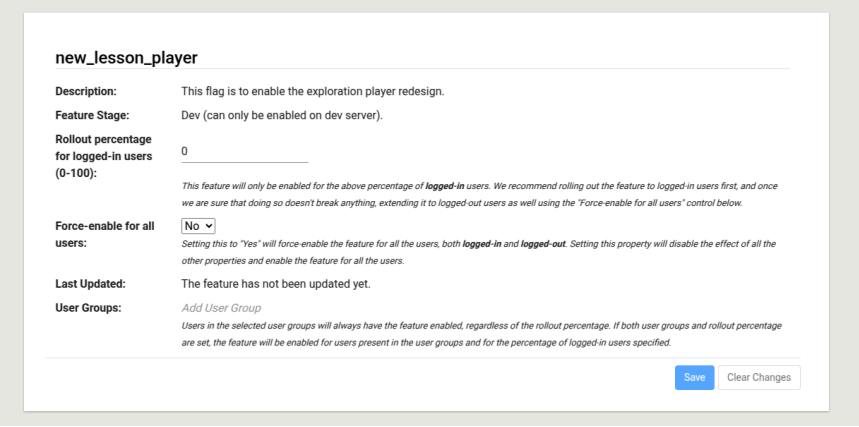


	Lesson player	**Practice question player**	**Diagnostic test player**	**Iframe**
Header	For curated lessons: Chapter {{chapter_umber}}: {{ chapter_name }} For community lesson: {{ Exploration_title }}	Practice Session: {{ Topic_name }}	Diagnostic Test: {{ classroom_name }}	N/A
Progress-bar (**Note:** These are not a common component. It's just a parameter for telling the difference. All the 3 things will be in different components/files.)	This includes a checkpoint bar at the footer.	This includes a progress bar below the header.	This includes a progress bar below the header.	N/A

Background-skin	Bluish-colored skin	Orange colored skin	There is no design available for this, so we will use the orange skin for this as well.	Bluish-colored skin
Footer	card-navigation-controls + lesson-player footer	card-navigation-controls + practice-question-player footer	card-navigation-controls	card-navigation-controls
Hints, Solution	Hints and solutions are present.	Hints and solutions are absent.	Hints and solutions are absent.	Hints and solutions are present.
Previous Card	We can visit the previous card.	We can visit the previous card.	We can't visit the previous card.	We can visit the previous card.

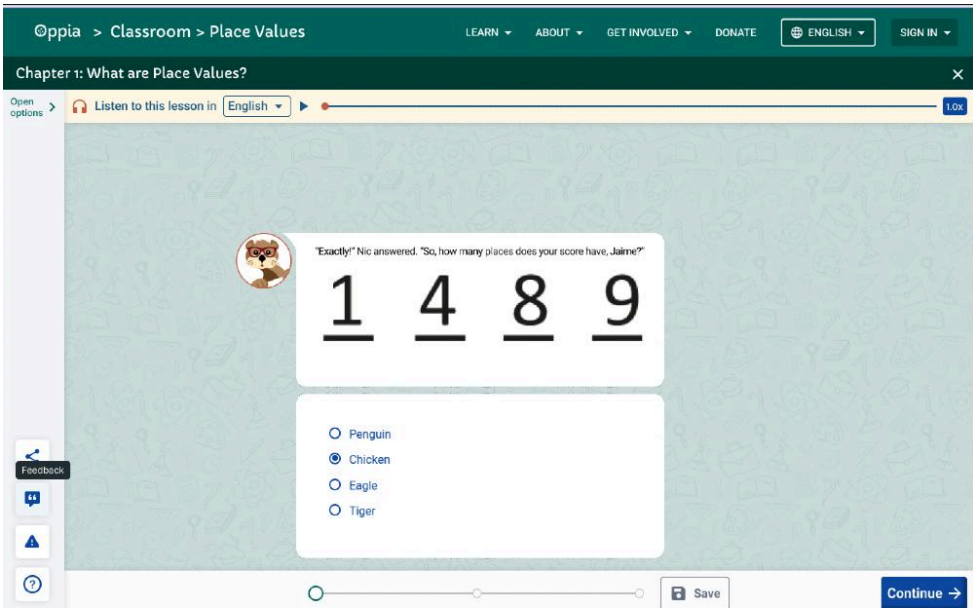
3. **CSS conventions**. We will create a separate CSS file for each new component (reference)

4. Using feature flag:
   - I will use the "new_lesson_player" feature flag, which already exists in the codebase and was introduced in this PR.

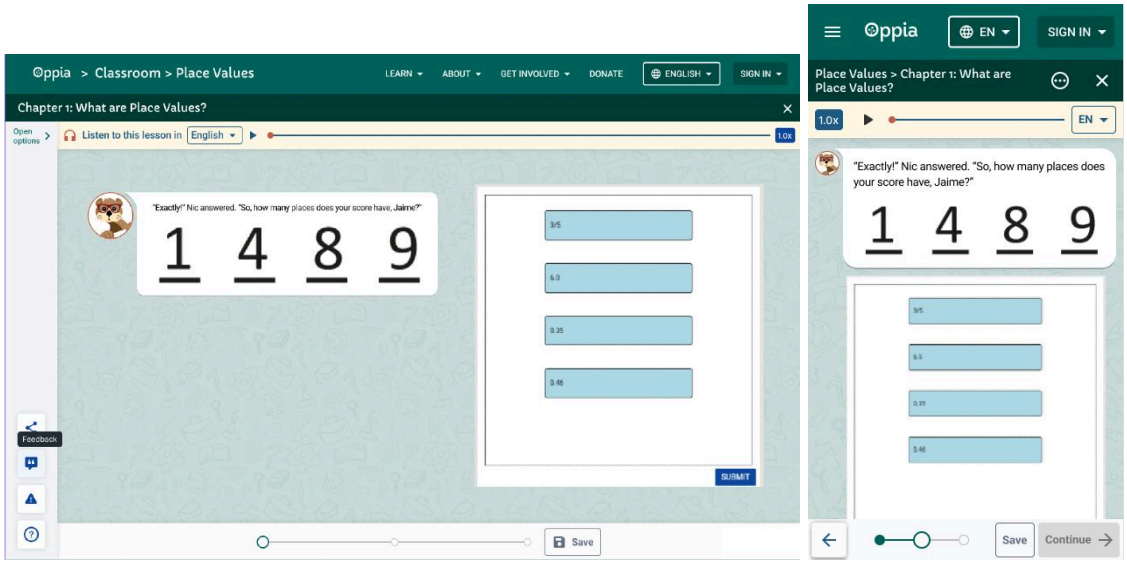


   - This feature flag already gates the /lesson route. This flag will also gate the new conversation skin for the diagnostic test player, practice question player and exploration preview.
   - "Behave correctly" specifies that all the new functionality developed should be hidden behind the feature flag, i.e, it should be only accessible when the feature flag is turned on.

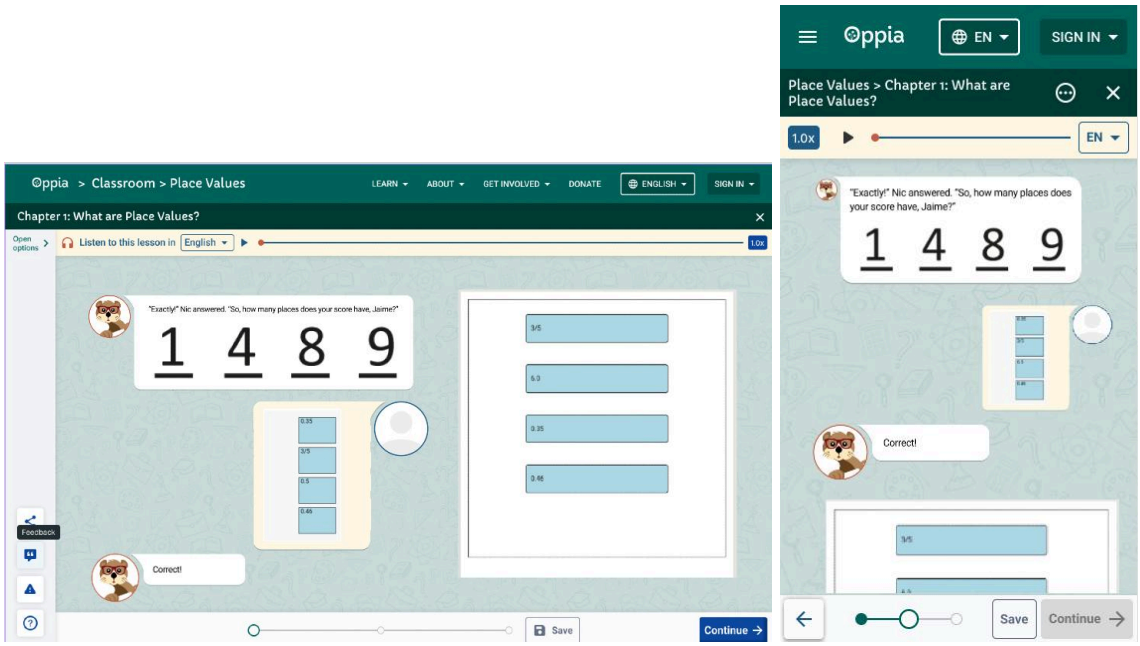
5. Connecting new interaction design -
   - We are not redesigning the interaction itself; only the skin and the conversation display component are being redesigned. Below is the mock for the Multiple Choice Interaction:



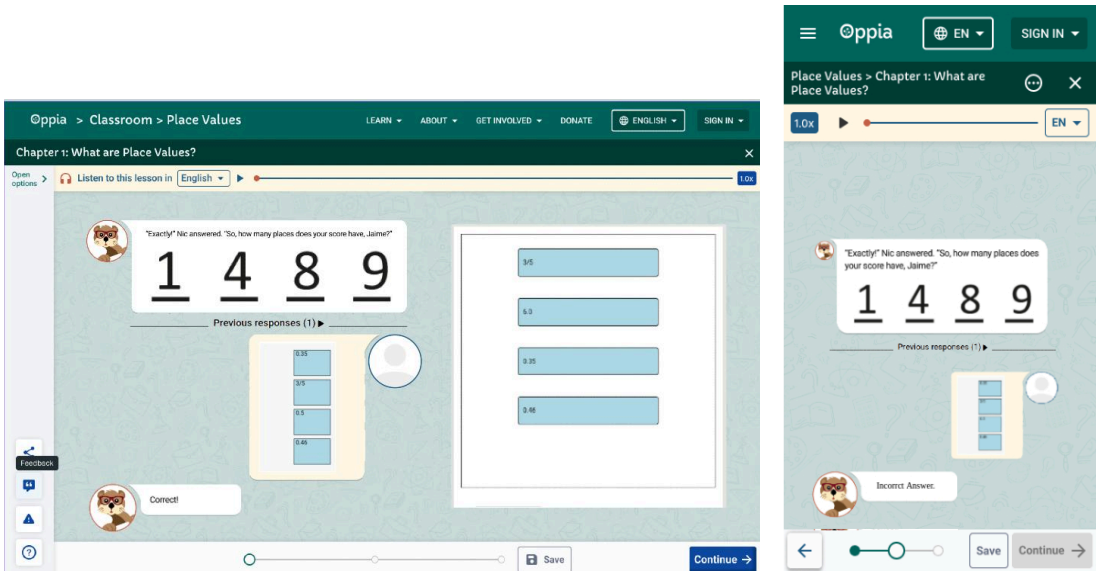
6. Handling supplemental interactions -
   - Supplemental interactions will be handled in the same way as they are currently done in the lesson player (reference). The mock below illustrates the initial state of the supplemental interaction, i.e, when the card is loaded completely.



Below is the flow that occurs when a learner submits the correct answer and receives feedback.



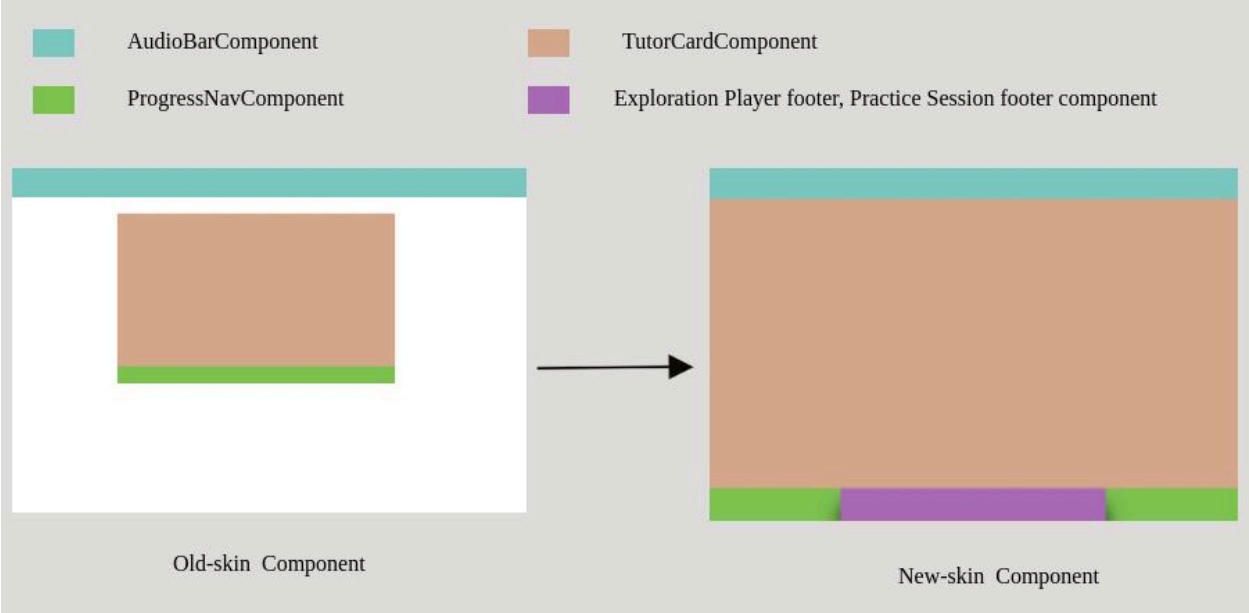
Finally, here is the flow for when a learner submits a wrong answer and receives feedback, followed by submitting the correct answer and receiving feedback.



7. The iframe will be managed by the new-conversation-skin component. There are a few key points to ensure:

- The sidebar should be hidden.
- The lesson-player-footer should be hidden, but the cardNavigationControls component (reference) should remain visible.
- We will use `this.urlService.isIframed()` to determine whether the lesson player is running inside an iframe.

8. Conversation Skin Component:



Third-Party Libraries

No new third-party libraries are required.

“Service” Dependencies

This feature adds no new service dependencies.

Key High-Level and Architectural Decisions

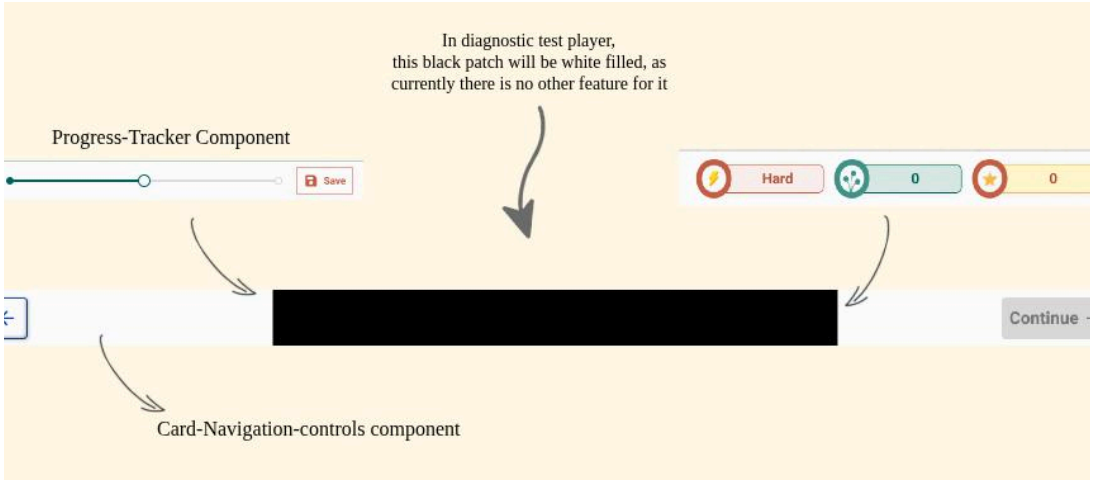
Decision 1: Development of footer

We have considered the following alternatives:

1. Developing a separate footer for every page.
2. Developing a nested structure for footers that can be commonly used.

Among these, I believe that second is the best approach, because:

- We can use a card-navigation-controls.component (Previously known as Progress-Navgiavtion component) for all the players, which will also serve as the footer when no additional functionality is present in the player’s footer, such as a diagnostic test.



- `Card-navigation-controls` will handle the user flow within the lesson, including actions like moving one card forward, moving one card backward, and submitting answers for non-text interactions (excluding supplemental interactions). It will check the mode once in the `ngOnInit` method and store the mode in a variable, allowing the rendering of respective components based on the mode in the template.
- `Progress-tracker.component.ts` will provide the UI for saving progress, interacting with related modals, and triggering checkpoint celebration events.

The above approaches are contrasted in detail in the following table:

	Developing a separate footer for every page.	**Developing a nested structure for footers that can be commonly used.**
Code reusability	No	Yes
Maintainable	No	Yes
Separable code logic	Progress navigation and footer functionality logic will be in the same component.	In this structure, the progress navigation will be in a separate component file, distinct from the footer functionality logic.
TS file	Separate TS file for each player or feature, therefore we need to duplicate the logic of the card navigation control component for every component.	One TS file for every feature or player, i.e, using the <ng-template> to render the child components.
Card-Navigation control logic	Duplicate code for each feature	No duplicate logic

Decision 2: Organizing Reusable Components

We have considered the following alternatives:

1. Separate the components that will not be used in the new lesson player from the reusable components.
2. Individual components for both the lesson players, no sharing of the components.

Among these, I believe that second option is the best approach, because:

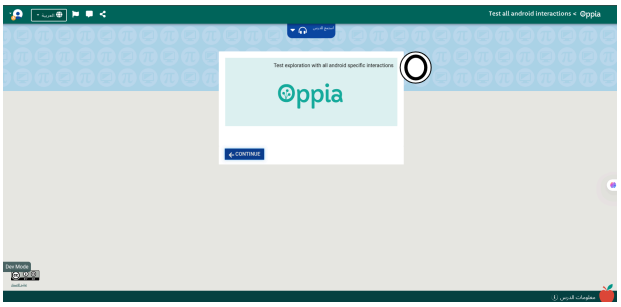
- There won't be any legacy code.
- Easier to remove the old code when the new lesson player is launched.
- Risk of regression will also be low.

The above approaches are contrasted in detail in the following table:

	Reusable Components	**Individual components for both lesson players**
Bug Fixing Efficiency	One fix applies to both players	Must fix bugs in both folders separately
Component Flexibility	Limited – Shared components need to support both use cases	High – Each player can evolve freely without affecting the other
Consistency Between Players (Tracking of bug)	Ensured automatically via shared components	If a contributor fixes a bug and unknowingly doesn't fix it for the other lesson player and issue gets closed, so it will leave an inconsistency for the new lesson player.
Risk of Regression	High – Fixes or changes can accidentally break both players	Low – Each player is isolated, so changes don't affect the other
Ease of Deleting Old Code Later	Harder – Shared components need cleanup or refactoring	Easy – Simply delete old player components and keep the new ones
Code Duplication	No	Yes

Risks and mitigations

Potential Risk	**Mitigation**
Users might find it difficult to find the back button or the continue button as they are positioned at the bottom corners.	Make the progress navigation component sticky at the bottom so that it should be always visible to the user, even when they scroll up.

User might get confused when they change the site language to some other language in which content is not translated, as it will stay default as English.	Show a modal that says “Currently the lesson is not translated in {{Selected Language}} so, the lesson will be shown in English”. Image below represents the situation where the lesson is not translated into any other language, but the user selects a RTL (Right-to-Left) language. In this case, the content will remain in the default English, while the page component will be translated into English. 
Iframes functionality can get skewed while refactoring the skin-component.	Provide video proof after every component redesign PR, that iframe is working fine.
Moving the logic from the component to services may break existing functionality, potentially leading to errors in production.	The following steps should be followed: • Any code moved from the component to the service file must maintain its original behavior—this includes returning or producing the same data and side effects as before the refactor. • New tests should be written for the service methods to verify they function identically to the original logic and are resilient to future changes.

Implementation Approach

Changes/Migrations to Storage Model Layer or Domain Objects

N/A

User Flows (Controllers and Services)

Note:

- This is a high-level overview of the key services used for the lesson player. For a more detailed description, please see the Appendix.
- **Implementation principle:** When migrating the services, any side effects must be obvious from the function name.

Handling of `visitedStateNames` Across Services

Currently, `visitedStateNames` is maintained in two separate places: `explorationEngineService` and `conversation-skin.component.ts`. Although both use the same variable name, they serve different purposes and are implemented differently:

- In `conversation-skin`, the variable is intended to store the **first visited state** and any **checkpoint states**, but due to a bug, the `firstState` is always `undefined`, and the logic does not consistently track checkpoint visits.

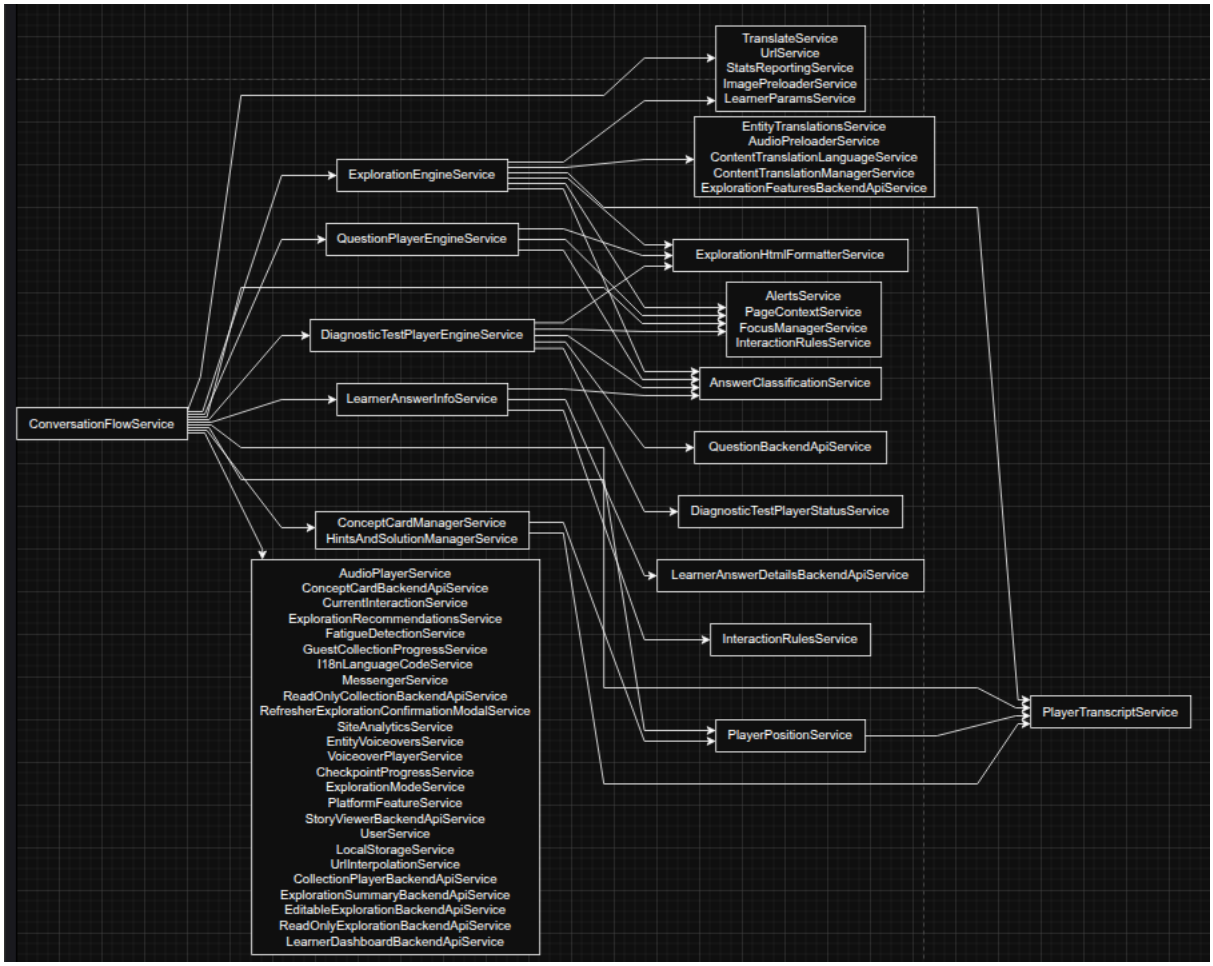
- In `explorationEngineService`, the goal is to maintain a list of **all visited states**, but this logic is also flawed. It incorrectly adds states even if they haven't been truly visited (e.g., just submitting a correct answer adds a state), and it fails to register the state when starting a lesson from a checkpoint.

In milestone 1.1 of the project, I will fix these issues **within their current implementations**, without merging them. This is because the two variables store **different types of data** and are used for **different purposes**, despite having the same name. A proper unification into a **single source of truth**, possibly via a method like `getVisitedStateNames()` in `PlayerTranscriptService`, is planned for **Milestone 1.4**.

Update `ConceptCardManagerService`:

- Remove the dependency on `ExplorationEngineService` (it's incorrect for this service to depend on it).
- Refactor it to behave like `HintsManagerService`.
- Eliminate the `conceptCardForStateExists` function.
- Move its logic into the `reset()` function instead.

The Image below shows the service dependency graph:



1. AudioPlayerService

Handles voiceover playback in lesson player. Supports play/pause, seeking, and language switching, using Howl for audio management.

Responsibilities: Control playback, track state, handle language changes

User Actions: Play/pause, seek, change voiceover language

2. ConversationFlowService

Manages learner flow across feature modes (exploration, questions, diagnostics). Coordinates card transitions, answer submission, and inactivity handling.

Responsibilities: Control progression, trigger state changes, manage “stuck” logic

User Actions: Submit answers, skip questions, click “Continue” or “Learn Again”

3. ProgressUrlService

Generates and manages a unique progress URL ID for logged-out learners to save and share progress.

Responsibilities: Create/fetch unique ID, save progress, provide progress URL

User Actions: None (background service)

4. CheckpointProgressService

Tracks learner progress through checkpoints in an exploration. Manages total/completed checkpoints and records the last save point.

Responsibilities: Track checkpoints, update progress, mark reached states

User Actions: None (background service)

5. ExplorationEngineService

Handles core logic for exploration progression: rendering states, submitting and classifying answers, managing transitions, and preloading media. Used specifically in exploration mode.

Responsibilities: Render states, classify answers, update state, preload media

User Actions: None (internal service used by ConversationFlowService)

6. PlayerPositionService

Tracks the learner’s current position in the card sequence and manages navigation between cards.

Responsibilities: Manage displayed card index, handle navigation, trigger updates

User Actions: Move to next card, move to previous card

7. ExplorationPlayerModeService

Tracks the current mode of the exploration player (exploration, pretest, questions, diagnostics, story chapter) and provides mode-specific checks.

Responsibilities: Identify current player mode, determine presentation type

User Actions: None (used internally)

8. PageContextService

Determines the current context of the page is running (e.g., blog page, about page).

Responsibilities: Identify page context (e.g., blog, topic, classroom)

User Actions: None (used for internal condition checks)

9. HintsAndSolutionManagerService

Manages the release and visibility of hints and solutions during a learner session. Controls hint/tooltips timing and tracks learner usage to emit relevant "stuck" or "solution viewed" events.

Responsibilities: Release hints/solutions, emit stuck/hint-consumed events, manage tooltip delays

User Actions: None (triggered by internal learning state and wrong submissions)

10. PlayerTranscriptService

Tracks the full learner session transcript — including cards shown, answers submitted, feedback, and hint/solution usage. Supports replay, progress restoration, and analysis. Also incorporates functionality from NumberAttemptsService.

Responsibilities: Log cards, answers, hints/solutions; provide transcript/history info; count attempts

User Actions: None (service is data/log focused)

11. QuestionPlayerEngineService

Handles core logic for practice sessions in Question Player. Manages question list, answer submission, classification, and feedback. Coordinates question transitions and UI rendering.

Responsibilities: Initialize/randomize questions, process answers, manage progression, render content.

User Actions: None (internal engine logic only)

12. DiagnosticPlayerEngineService

Manages diagnostic test progression logic. Handles question fetching, answer processing, topic/skill transitions, and learner progress tracking. Used exclusively in diagnostic test mode.

Responsibilities: Initialize session from tracker model, classify answers, transition across topics/skills, track attempts, emit progress/completion events.

User Actions: None (all interactions handled internally)

13. UrlService

Utility service for parsing and manipulating URLs. Used across the app to extract parameters like topic/story/classroom IDs, and build or modify query strings.

Responsibilities:

- Parse query parameters and hash fragments
- Extract identifiers from learner and exploration URLs
- Construct URLs with additional fields

User Actions: None (all logic is internal, supporting routing and configuration)

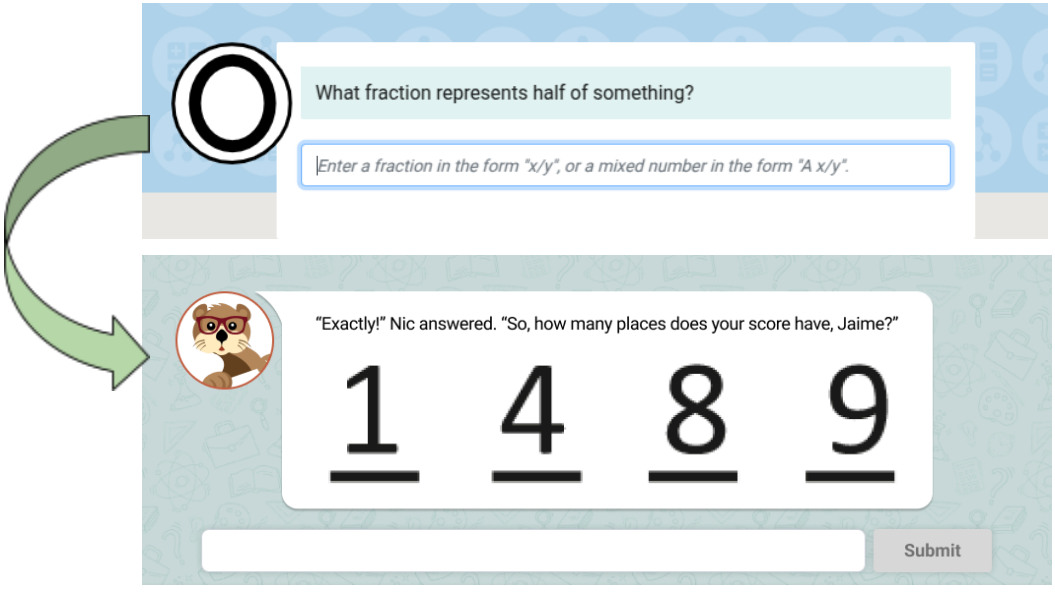
[Web only] Web frontend changes

Note: The changes documented below affect UI that is shared across multiple features. Therefore, it is important to implement them carefully to ensure correctness and consistency across all affected features.

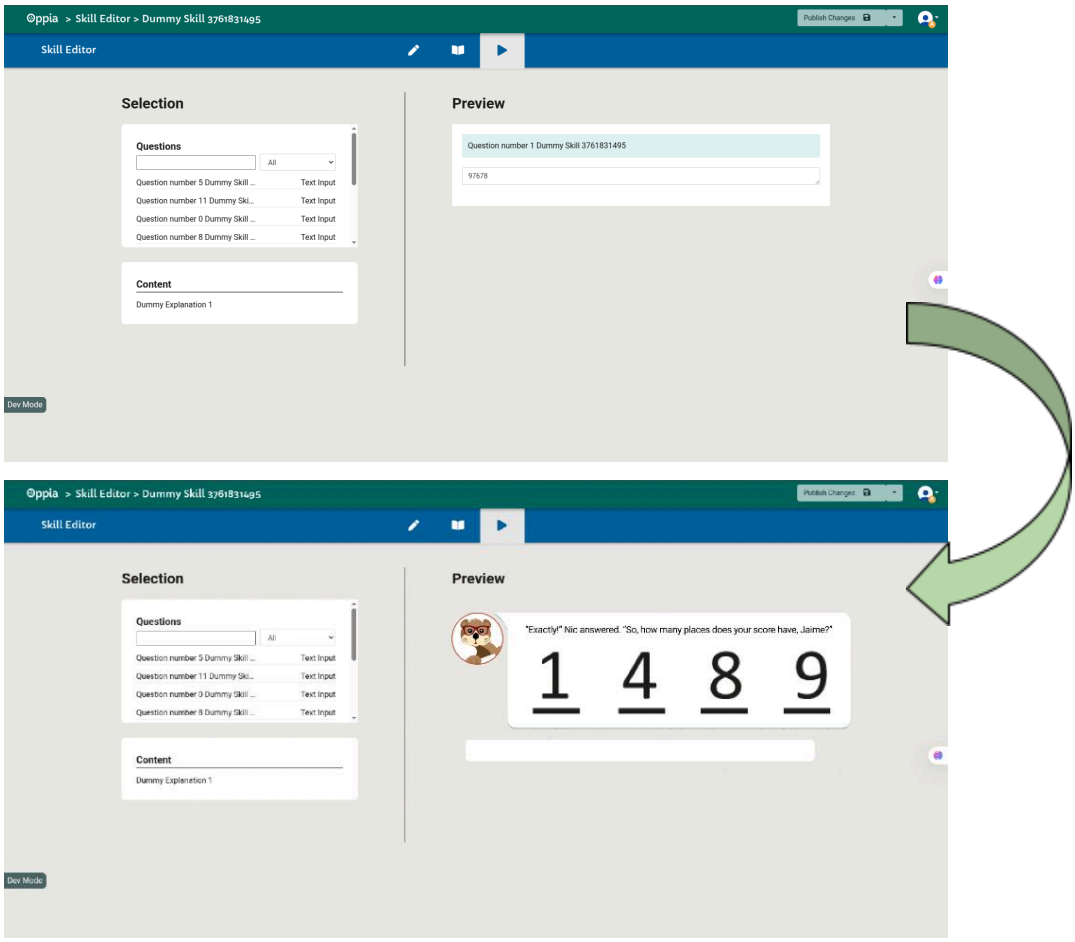
1. Tutor Card -> Conversation display

Tutor card component is used to display the question and the input field of the displayed card. This component is used in 2 components:

- Conversation-skin.component



- Skill-preview-tab.component



2. Checkpoint bar:
   - The following will be the methods in the new checkpoint bar component:
     - getCompletedProgressBarWidth()
     - getProgressPercentage()
     - getMostRecentlyReachedCheckpointIndex()
     - updateLessonProgressBar()
     - getCheckpointCount()

3. Checkpoint Celebration:
   - Animations:
     - The animation logic for the text box is based on the same approach used in CorrectnessFooterComponent.
     - The animation for the checkpoint bar reuses logic from CheckpointCelebrationModalComponent (Specific CSS line to check on are #180 - #190).
   - Confetti:
     - The confetti animation reuses the logic from the existing EndChapterConfettiComponent.

4. Fix navbar:

- Home_tab.png : As part of the new lesson player, we will be replacing the legacy custom navbar (previously used for reporting, feedback, etc.) with the standard site-wide navbar, as reflected in the new design mocks (Figma link).
- Given this change, the logic for hiding the "Home" tab for logged-in users should also be applied in this context to maintain consistency with the rest of the site. While initially considered a minor edge case, the adjustment is straightforward (approximately 10–15 lines of code) and helps ensure a cohesive user experience.
- For this reason, this task will be handled as part of **Milestone 2.6**.

5. Progress-Tracker Component:

This component is somewhat tricky to implement because it is embedded within another component and needs to behave as a **dynamic component** based on the current feature mode. To handle this behavior, we use the code shown below:

TS file

```
@ViewChild('dynamicComponent', { read: ViewContainerRef }) container!: ViewContainerRef;

ngOnInit() {
if(!iframe && this.mode === isInExplorationPlayerPage()){
  this.container.createComponent(component);
}}
```

Template file

```
<ng-template #dynamicComponent></ng-template>
```

6. Organising Modals:

- All modals in the codebase are feature-specific, so there's no need for shared or reusable modals across pages. Based on that, we'll use the following approach:
    - **Keep modals with their features**: Modals stay alongside the components and logic they belong to. This keeps each feature self-contained and easy to maintain.
    - **No shared modal folder**: Since we're not using shared modals, a general folder isn't necessary.

Documentation changes

N/A

Metrics Plan

Event (see PRD)	**Event parameters (see PRD)**	**Do we already record the event + parameters?**
audio_played	• explorationId • card_number	Yes (code)

share_exploration	• network	Yes (code)
open_embed_info_modal	• explorationId	Yes (code)
visit_oppia_from_iframe	• explorationId	Yes (code)
lesson_started	• explorationId	Yes (code)
lesson_completed	• explorationId	Yes (code)
answer_submitted	• explorationId • answerIsCorrect	Yes (code)
new_card_load	• cardNum • explorationId	Yes (code)

Implementation Plan

Note: With each PR, proof that the following functionality works fine should be attached:

- Iframe
- New Lesson player
- Old lesson player

Milestone 1

Key Objective
The following actions can be performed when the feature flag is turned on: • Visit /lesson route and see the new lesson player layout • Submit and view wrong answers • Navigate back and forth through the lesson • See pop-up and confetti when the correct answer is submitted. • New skin-component UI in the practice player, exploration editor(preview tab), and in diagnostic test pages • User will be redirected to /lesson route, if he visits the /explore route.

No.	Description of PR / action	Prereq PR numbers	Target date for PR creation	Target date for PR to be merged	Demo date
1.1	Fix the visitedStateNames variable bug **Goal:** Fix inconsistencies in how the `visitedStateNames` variable is handled across the `explorationEngine` service and the	-	11th June	13th June	

	`conversation-skin` component. The goal is to ensure it accurately tracks visited states regardless of whether the exploration is started from the beginning or from a checkpoint. **Completion:** In `conversation-skin`, fix the issue where `firstState` is always `undefined`, and ensure that only valid checkpoint-related states are stored. In `explorationEngine`, fix the bug where new states are added even if not visited, and ensure that the initial state is correctly captured when starting from a checkpoint. After the fix, `visitedStateNames` accurately reflects all visited states in order, and behaves consistently across both fresh sessions and resumed ones via checkpoints. **Note:** This milestone won't migrate any state to other file, it will be just fixing the bug. This state will be shifted in the milestone 1.4				
1.2	Refactor the folder structure and add README files **Goal:** Organize all files in the `exploration-player-page` folder according to the new folder structure specified in the solutions section. **Completion:** All files are moved to match the updated blueprint. No file contents are modified except for updating import paths to reflect their new locations. README files are added to relevant directories to document their purpose. No functional or styling changes are made—this is strictly a structural refactor. **Tests to ensure feature flag correctness:** • Verify that the `/lesson/<lesson_id>` route is inaccessible when the feature flag is disabled.	1.1	11th June	13th June	

	• Verify that the route loads correctly and functions as expected when the feature flag is enabled.				
1.3	Create `ExplorationModeService`, dismantle `ExplorationPlayerStateService`, and rename `ContextService` to PageContextService. **Goal:** Introduce a new `ExplorationModeService` to manage exploration modes (story mode, question mode, editor preview, etc.). Progressively dismantle the legacy `ExplorationPlayerStateService` by migrating its responsibilities into appropriate, modular services. Finalize the renaming of `ContextService` to `PageContextService`, and remove any functions unrelated to page-level context—such as those tied to exploration mode or other responsibilities that belong in separate services. **Completion:** `ExplorationModeService` is created and handles all logic specific to exploration modes. `ExplorationPlayerStateService` is fully dismantled, with its logic redistributed appropriately. `ContextService` is renamed to `PageContextService`, and all unrelated functions (e.g., mode-checking utilities) are removed or moved to the corresponding services.	1.2	12th June	14th June	
1.4	Migrate unnecessary variables from the conversation-skin component **Goal:** Move state-related variables that do not need to reside in the `ConversationSkinComponent` into their relevant services to improve separation of concerns and reduce component complexity.	1.3	13th June	16th June	

	Completion: The following variables are migrated from `ConversationSkinComponent` to their appropriate services: • **`answerIsBeingProcessed`** → Moved to `ConversationFlowService` (manages state of ongoing answer submission). • **`nextCard`** → Moved to `ConversationFlowService` (manages progression logic). • **`nextCardIfStuck`** → Moved to `ConversationFlowService` (used in stuck learner logic). • **`visitedStateNames`** → Moved to `PlayerTranscriptService` (tracks all visited states for the learner). • **`completedStateNames`** → Moved to `PlayerTranscriptService` (manages record of completed states). • **`mostRecentlyReachedCheckpoint`** → Moved to `CheckpointProgressService` (tracks the last checkpoint reached). • **`numberOfIncorrectSubmissions`** → Moved to `PlayerTranscriptService` (tracks incorrect attempts per state). Each variable's logic is migrated cleanly, and `ConversationSkinComponent` now interacts with them through well-defined service APIs. No functionality is broken during or after				

	the migration.				
1.5	Decouple back and forth logic from the `conversation-skin` component and create a public API in the existing PlayerPositionService **Goal:** Extract all navigation logic (moving forward and backward between cards) from the `ConversationSkinComponent` and expose it via a clean, reusable public API in the existing `PlayerPositionService`. **Completion:** All logic related to navigating forward and backward between cards is removed from the `conversation-skin` component and refactored into a dedicated service. A clear and well-documented public API is created in the service to handle these operations.	1.4	14th June	17th June	
1.6	Finalize the `ExplorationEngineService`, move non-related functionality into respective services (`DiagnosticTestEngineService`, `QuestionPlayerEngineService`, `ConversationFlowService`) **Goal:** Streamline the `ExplorationEngineService` by ensuring it only handles responsibilities related to managing exploration mode flow and state transitions. Relocate any unrelated logic—such as diagnostic test handling, question player logic, or conversation-specific behavior—into their appropriate dedicated services: `DiagnosticTestEngineService`, `QuestionPlayerEngineService`, or `ConversationFlowService`. **Completion:** All logic in `ExplorationEngineService` unrelated to core exploration flow is identified and migrated to their respective services (`DiagnosticTestEngineService`,	1.5	15th June	18th June	

	`QuestionPlayerEngineService`, or `ConversationFlowService`). The `ExplorationEngineService` is left with a focused scope, handling only exploration state transitions and progression logic				
1.7	Migrate logic of complex functions like `submitAnswer`, `moveToCard`, `addNewCard`, etc. into `ConversationFlowService` **Goal:** Improve code modularity and maintainability by extracting complex and multi-responsibility functions such as `submitAnswer`, `moveToCard`, `addNewCard`, and similar logic-heavy methods from the component and moving them into the `ConversationFlowService`. **Completion:** The core logic of functions like `submitAnswer`, `moveToCard`, `addNewCard`, and other related methods is migrated from the `ConversationSkinComponent` into the `ConversationFlowService`. Each function is refactored as needed to fit within the service's responsibility boundaries (e.g., answer processing, navigation, and card management). The component delegates these responsibilities through a clean, well-documented public API exposed by `ConversationFlowService.`	1.6	17th June	20th June	
1.8	Migrate all remaining logic from the `ConversationSkinComponent` and dismantle `QuestionPlayerStateService`. **Goal:** Complete the migration of all leftover conversation flow logic out of the `ConversationSkinComponent` and into appropriate services with well-defined responsibilities. Fully dismantle the now-obsolete `QuestionPlayerStateService.` **Completion:** All remaining conversation flow logic is moved from components to dedicated services with clear responsibility boundaries.	1.7	19th June	21st June	

	The `QuestionPlayerStateService` is completely dismantled and removed from the codebase. All dependent code is updated to use the new service structures. All remaining logic in `ConversationSkinComponent` related to conversation flow is identified and moved into relevant services such as `ConversationFlowService`, `PlayerTranscriptService`, or others as appropriate. The `QuestionPlayerStateService` is fully dismantled and removed from the codebase. All dependencies and references to the dismantled service are updated to use the new service structure.				
1.9	Get mentor + TL review (of the current state of the service architecture in the codebase). Make any remaining changes needed.	Up to 1.8	21st June	23rd June	
1.10	Create new conversation skin component, create conversation display component (only first card shown) **Goal:** Create a new conversation skin component for the exploration player and develop a conversation display component that shows only the first card. Build the conversation display component to support all the RTE components, all the interactions, and the supplemental cards. **Completion:** The new conversation skin component is implemented and styled. The conversation display component renders only the first card on load. All components are connected smoothly with no visual or functional issues. Write acceptance tests for CUJs (EL.LP) (EL.HC [1.1]) **Note:** Lesson player component is not	1.9	22nd June	25th June	

	same as of the conversation skin component. Lesson player component contains the conversation skin component and it is only used in the lesson player, whereas conversation skin component will be used in the different feature modes like the practice question player, iframe, etc. **Tests to ensure feature flag correctness:** • Confirm that flag-based routing behaves consistently across different features, new conversation skin should not be visible in the practice question player and diagnostic test player, if the feature flag is off.				
1.11	Create new `CardNavigationControlComponent` with forward/backward navigation, continue, and skip question functionality in the Diagnostic Test Player **Goal:** Develop a new, reusable `CardNavigationControlComponent` that allows users to navigate forward or backward through the lesson, continue to the next step, or skip a question—specifically within the Diagnostic Test Player context. **Completion:** The `ProgressNavComponent` is implemented and fully functional. It supports forward and backward navigation, continue progression, and skip question functionality. The component is integrated with the appropriate services (e.g., `DiagnosticTestEngineService` and `PlayerPositionService`) Write acceptance tests for CUJs (EL.HC [1.3][1.4])	1.10	24th June	27th June	
1.12	Implement Answer submission,	1.11	26th June	29th June	

	feedback & response handling, **Goal:** Implement answer submission flow, handle feedback and response display, and enable skip question functionality within the exploration player. **Completion:** Answer submission processes user responses correctly and triggers appropriate feedback. Feedback and responses are displayed clearly and in a timely manner. Skip question functionality works seamlessly without breaking the flow or state consistency. Write acceptance tests for CUJs (EL.HC [1.5] [1.7])				
	Demo to PMs/stakeholders	Up to 1.12			2 July
1.13	Bug fixes for milestone 1 (if required)			<10 July	

Milestone 2

Key Objective
The following actions can be performed when the feature flag is turned on: ● Can view hint, solutions and concept cards ● Voiceover is audio can be played ● Progress saving and checkpoints flow ● Rate the chapter and get recommendations at the end of the lesson. ● All the modals from the sidebar can be viewed Once all the feature are developed: ● Remove feature flag ● Remove the dead code

No.	**Description of PR / action**	**Prereq PR numbers**	**Target date for PR creation**	**Target date for PR to be merged**	**Demo date**
2.1	Integrate hints, solutions and linked concept cards functionality in conversation display component and modals	Upto 1.13	26th July	30th July	

	Goal: Integrate hints, solutions, and linked concept cards into the conversation display component, and displaying their modals as needed. **Completion:** The conversation display component now supports hints, solutions, and linked concept cards, all of which are displayed correctly in modals. The interaction is smooth, and the features are fully integrated without errors. Write acceptance tests for CUJs (EL.HC [1.2] [1.6])				
2.2	Create share modal **Goal:** Develop a modal for sharing content from the exploration player. This modal should allow users to copy a shareable link and be accessible from the sidebar. **Completion:** The Share modal is designed and fully functional. It is integrated into the sidebar and can be opened via a clearly visible trigger. The modal displays a correctly formatted shareable link and provides a copy-to-clipboard feature. Write acceptance tests for CUJs (EL.SL)	-	30th July	2nd Aug	
2.3	Implement Lesson End Celebration: Confetti, Checkmark Animation, and Correct Answer Submission Popup **Goal:** Enhance the user experience at key milestones by implementing a celebration animation when a lesson ends, animating a checkmark for completed cards, and displaying a popup for correct answer submissions. **Completion:** ● A confetti animation is triggered upon lesson completion. ● A checkmark animation plays when a user successfully	-	2nd Aug	5th Aug	

	completes a card. • A popup appears immediately after a correct answer submission, confirming correctness and guiding the user to proceed. All animations and UI feedback are integrated into the relevant components and services (e.g., `ConversationFlowService`). Interactions are smooth, non-blocking, and responsive across devices.				
2.4	Implement Lesson Language Change Logic Based on Site Language **Goal:** Enable users to change the lesson language dynamically, ensuring that content updates correctly and user progress is handled gracefully. **Completion:** Language change logic is implemented such that when the site language changes, the corresponding lesson content updates to reflect the selected language. If the user attempts to change the language mid-lesson, a confirmation modal appears warning that progress may be lost. Upon confirmation, the lesson is reloaded in the new language.	-	4th Aug	7th Aug	
-	**Note:** At this point, Embedded Exploration (Iframe) functionality will be ready with the new redesign. All the user journeys can be tested by turning on the NEW_LESSON_PLAYER feature flag.				
2.5	- Create common header component **Goal:** Develop a reusable header component to be used across the application. **Completion:** A fully functional, reusable header component is created and integrated where applicable without breaking existing functionality. **Tests to ensure feature flag**	-	6th Aug	9th Aug	

	correctness: • Confirm that flag-based routing behaves consistently across different features, header should not be visible in the practice question player and diagnostic test player, if the feature flag is off.				
2.6	Refactor diagnostic test and practice question player design **Goal:** Redesign the diagnostic test and practice question player interfaces to enhance user experience. **Completion:** The designs for both the diagnostic test and the practice question player are updated, with improved usability and consistent styling, while maintaining current functionality.	2.5	7th Aug	9th Aug	
-	**Note:** At this point, new design for the Practice question player and the diagnostic test player functionality will be ready. All the user journeys can be tested by turning on the NEW_LESSON_PLAYER feature flag.				
2.7	Create Audio-bar component **Goal:** Implement the functionality of the audio-bar component to support media playback controls such as play, pause, seek. **Completion:** The audio-bar supports audio playback features including play, pause, and progress tracking. It integrates seamlessly with the lesson or media content where applicable. Write acceptance tests for CUJs (EL.VO)	-	9th Aug	12th Aug	
-	**Note:** At this point, new design for the Exploration Editor's Preview mode will be ready. All the user journeys can be tested by turning on the NEW_LESSON_PLAYER feature flag.				
2.8	- Create feedback modal - Create Report modal - Integrate the above modals in the sidebar	-	12th Aug	14th Aug	

	Goal: Develop modals for submitting feedback and reporting issues, and integrate them into the exploration player's sidebar for easy access. **Completion:** The Feedback, and Report modals are designed, fully functional, and integrated into the sidebar. Users can open these modals from the sidebar, and they capture necessary user inputs (feedback submission, and issue reporting) correctly. Write acceptance tests for CUJs (EL.FB)				
2.9	- Create Common Checkpoint bar component - Create new Exploration-footer **Goal:** Create reusable components for the checkpoint bar and exploration footer that can be used across different pages. **Completion:** The checkpoint bar is created, dynamically reflecting the user's progress, and the exploration-footer is designed and functional across supported screen sizes and devices.	-	14th Aug	17th Aug	
2.10	- Implement Checkpoint celebration logic - Create Save-progress modal **Goal:** Implement logic to celebrate the completion of a checkpoint and create a save-progress modal that allows users to save their progress at any time. **Completion:** The checkpoint celebration logic works as expected (e.g., displaying an animation or message upon reaching a checkpoint). The save-progress modal is functional, and progress is saved when the user confirms.	2.9	16th Aug	19th Aug	
2.11	-Create progress-reminder modal -Create exit card modal **Goal:** Implement the progress-reminder modal to ask users whether to start the lesson from the saved checkpoint or	2.9	19th Aug	21rd Aug	

	from the beginning. Also, create the exit card modal to confirm the exit, offering options to save or discard progress. **Completion:** The progress-reminder modal is triggered when a user re-visits a lesson with some save progress, prompting users to choose whether to continue from the saved checkpoint or start over. The exit card modal appears when attempting to leave, providing the option to save or discard progress. Write acceptance tests for CUJs (LI.SP [1.4]) (EL.CP)				
2.12	Redesign Last Card **Goal:** Redesign the last card of the lesson to allow learners to rate the lesson, visit the next lesson, practice questions, or revise the topic. **Completion:** The last card is redesigned with options for lesson rating, proceeding to the next lesson, practicing questions, and revisiting the topic, providing a more engaging end-of-lesson experience Write acceptance tests for CUJs (EL.CE)	-	21st Aug	24th Aug	
	Demo to PMs/stakeholders:	Up to 2.12			26 Aug
2.13	Bug fixes for milestone 2 (if required)			< 5 Sept	
2.14	Remove the old code **Goal:** Removal of the old code of the lesson player and centralize the new lesson player code. **Completion:** /Exploration-player-plage folder should look like the below structure: /exploration-player - /sidebar-component - / conversation-skin-components - /lesson-header - /Services	Up to 2.13	5 Sept		

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.	new_lesson_player	- /explore route will be redirected to /lesson route. - New-lesson player can be accessed on the /lesson route.. - New design for the diagnostic-test and the practice-question player will be visible.	No, since this project only involves UI changes and no data changes are being made.	LaCe Team

Appendix – List of updated service descriptions/APIs

AudioPlayerService:

Voiceover playback service. This service manages the playback, control, and state of a voiceover audio file. It provides functionality to play, pause, seek, reset, and change language for the voiceover, along with loading audio files and querying playback state.

- What are the user interactions?
    - AudioBarComponent
        - When the user plays or pauses the audio.
        - When the user scrubbed to a position
        - Change voiceover language (different than translation)
- Which states do I need?
    - currentAudioTrack: Howl (this typically represents an audio track, and it internally handles things like the audio file URL (or blob), playback state (playing, paused, etc.), volume, looping, events, etc.)
    - currentAudioLanguage: string
        - This is initialized when the lesson player is initialized or the audiobar is initialized.
        - It is changed when a user changes the voiceover language from the dropdown.
    - lastPauseOrSeekPositionSecs: number. Needed in the following cases:
        - If a user pauses audio at a certain time and then needs to play the audio from that point onwards
        - If a user pauses audio at a certain time, and just manually scrubs to a position, while the audio is in the paused state, we need to store the new time instance that is scrubbed to
- What functions do we need?

```
/**
 * Called when the voiceover playback starts.
 * Initializes the current voiceover playback if it isn't already playing.
 * Resumes playback from the last known paused or seek position.

 */
```

```
function onPlay(): void {}
```

```
/**
 * Called when the voiceover playback is paused.
 * Implement this to handle any behavior that should occur on pause.
 */
function onPause(): void {}
```

```
/**
 * Called when the user seeks to a different position in the voiceover.
 *
 * @param {number} newVoiceoverPositionSecs - The new playback position in seconds.
 */
function onSeek(newVoiceoverPositionSecs: number): void {}
```

```
/**
 * Called when the voiceover language is changed.
 *
 * @param {string} selectedLanguage - The newly selected language code (e.g., 'en', 'es').
 */
function onVoiceoverLanguageChange(selectedLanguage: string): void {}
```

```
/**
 * Resets the voiceover to its initial state.
 * This should stop playback, clear the lastPauseOrSeekPositionSecs, set the voiceoverIsPlaying
to false, and prepare for a fresh start.
 */
function resetVoiceover(): void {}
```

```
/**
 * Loads a voiceover file for playback.
 *
 * @param {string} filename - The name or path of the voiceover file to load.
 * @returns {Promise<void>} A promise that resolves once the voiceover is successfully loaded.
 */
function loadVoiceover(filename: string): Promise<void> {}
```

```
/**
 * Checks whether a voiceover is currently playing.
 *
 * @returns {boolean} `true` if the voiceover is playing, otherwise `false`.
 */
function isVoiceoverPlaying(): boolean {}
```

ConversationFlowService:

Service responsible for orchestrating the flow of learner interactions across different player modes (lesson, question, diagnostic test, etc.). This service acts as the top-level coordinator for controlling the presentation of cards (questions or content), triggering answer submissions, providing feedback, and managing the learner's progression through the session. It delegates engine-specific logic (e.g., state transitions, answer evaluation) to mode-specific engine services:

- ExplorationEngineService for lesson player
- QuestionEngineService for question player
- DiagnosticEngineService for diagnostic tests

Core responsibilities include:

- Displaying new cards and managing the transcript view.
- Submitting learner answers and updating UI state.

- Maintaining the flow of interactions, including refreshes.
- Bridging player-specific engines with shared transcript, analytics, and UI logic.

- What are the user interactions?
    - ConversationSkinComponent
        - User actions through which we can redirect to the other card:
            - Click on the learn Again button (I am assuming this and below have similar use cases, but need to check in more depth to tell the actual difference between both).
            - Show the stuck state "**Continue**" button if the user is inactive for **150 seconds**, under the following conditions:
                - **When a new card is opened:**
                    - The 150-second timer starts.
                    - If hints are available but have not been consumed (or hints are shown), and the user hasn't answered the question, the "Continue" button is shown after the timeout.
                - **After all hints are consumed:**
                    - The timer resets and starts again.
                    - If the question remains unanswered when the 150 seconds expire, the "Continue" button is shown.
        - Submit Answer
        - Change the site language (if so, the lesson language should be changed (if available)).
        - Skip diagnostic test question
- Which states do I need?
    - **[Not stored in this service]** The player transcript from playerTranscriptService (used by this service, but not stored locally)
    - nextStateNameAfterLearnerReadsCorrectAnswerFeedback: string
        - It is the state that's ready to add to the stack (but we haven't added it yet, so the learner does not see it)
    - nextStateIfStuckName: string | null;
    - answerIsBeingProcessed: boolean = false;
- What functions do we need?

```
/**
 * Retrieves the currently active engine service, which could be one of
 * Exploration, Question Player, or Diagnostic Test engines.
 *
 * @returns {ExplorationEngineService | QuestionPlayerEngineService |
DiagnosticTestPlayerEngineService}
 * The active engine service instance.
 */
function getCurrentEngineService():
  | ExplorationEngineService
  | QuestionPlayerEngineService
  | DiagnosticTestPlayerEngineService {}
```

```
/**
 * Submits the learner's answer for processing.
 *
 * This method first checks whether the answer can be submitted, i.e, to prevent
 * answer spamming, it may emit an event to display the "Take a Break"  modal.
 *
 * If submission is allowed, it performs any necessary pre-submission logic.
 * If additional input from the learner is needed (e.g., clarification or follow-up),
```

```
 * the method requests it and exits early.
 * Otherwise, the answer is submitted to the exploration engine for evaluation.
 *
 * @param {string} answer - The learner's submitted answer.
 * @param {InteractionRulesService} interactionRulesService - The service used to evaluate the
answer based on the current interaction's rules.
 */
function onSubmitAnswer(
  answer: string,
  interactionRulesService: InteractionRulesService
): void {}
```

```
/**
 * Skips the current diagnostic test question.
 */
function onSkipDiagnosticTestQuestion(): void {}
```

```
/**
 * Returns the event emitter that is triggered when the player state changes.
 *
 * @returns {EventEmitter<string>} The event emitter that emits the name of the new state.
 */
function get onPlayerStateChange(): EventEmitter<string> {}
```

```
/**
 * Appends a new response to the latest feedback shown on the last card.
 *
 * This is the case where a response gets added when the learner is stuck in the
 * feedback
 *
 * @param {string} response - The response text to add.
 */
function addNewResponseToExistingFeedback(response: string): void {}

Moved from playerTranscriptService
```

```
/**
 * Returns the event emitter that is triggered when a new card is opened.
 *
 * @returns {EventEmitter<StateCard>} The new card opened event emitter.
 */
function get onNewCardOpened(): EventEmitter<StateCard> {}
```

ProgressUrlService:

Generates and retrieves a unique progress URL ID for logged-out learners.

- What are the user interactions?
    - N/A
- Which states do I need?
    - uniqueProgressUrlId: string | null = null;
- What functions do we need?

```
/**
 * Returns the unique identifier for the progress URL.
 *
 * @returns {string} The unique progress URL ID.
 */
function getUniqueProgressUrlId(): string {}
```

Moved from ExplorationPlayerStateService

```
/**
 * Saves the user's progress while they are logged out by generating
 * a unique progress URL ID and constructing a shareable progress URL.
 *
 * If the unique progress URL ID is not yet set, it invokes the
 * `setUniqueProgressUrlId()` method
 * to generate it and constructs the full URL using the origin.
 *
 *
 * @returns {Promise<void>} A promise that resolves once the operation is complete.
 */
async saveLoggedOutProgress(): Promise<void> {
```

```
/**
 * Asynchronously requests and sets a unique progress URL ID for a logged-out learner.
 * This ID helps persist progress and is fetched from the backend.
 * After fetching, it triggers progress tracking.
 *
 * Dependent state variables:
 * - this.explorationId
 * - this.version
 * - this.lastCompletedCheckpoint
 * - this.uniqueProgressUrlId
 *
 * Dependent services:
 * - editableExplorationBackendApiService (fetches the unique progress ID)
 *
 * Side effects:
 * - Updates `this.uniqueProgressUrlId` with the backend response.
 * - Calls `this.trackLoggedOutLearnerProgress()` to initiate tracking.
 *
 * @function setUniqueProgressUrlId
 * @returns {Promise<void>} A promise that resolves once the ID is set and tracking is
triggered.
 */
async setUniqueProgressUrlId(): Promise<void> { ... }
```

CheckpointProgressService:

@fileoverview Service is used to manage learner checkpoints in an exploration.

This service handles the logic for tracking and recording learner progress through checkpoints. It provides methods to:
- Get total and completed checkpoint counts.
- Mark checkpoints as reached.
- Track and update the last completed checkpoint.

- What are the user interactions?
    - N/A
- Which states do I need?
    - totalCheckpointCount: number
    - completedCheckpointCount: number
    - checkpointStatusArray: string[]
    - mostRecentlyReachedCheckpoint: string;
        - the last save point (i.e. where to put the user when they reload the page)
- What functions do we need?

```
/**
 * Gets the total number of checkpoints.
 *
 * @returns {number} The total count of checkpoints.
 */
function getTotalCheckpointCount() {}
```

```
/**
 * Gets the number of checkpoints that have been completed.
 *
 * @returns {number} The count of completed checkpoints.
 */
function getCompletedCheckpointCount() {}
```

```
/**
 * Calculates and returns the index of the most recently reached checkpoint.
 *
 * Iterates through the cards in the player transcript and counts how many of them
 * corresponds to states that are marked as checkpoints. The index is determined
 * by the number of checkpoint cards encountered.
 *
 * @returns {number} The index of the most recently reached checkpoint.
 */
function getMostRecentlyReachedCheckpointIndex(): number {}
```

```
/**
 * Marks a checkpoint as reached for the given state and version.
 *
 * This updates the last completed checkpoint in the player's state service and
 * sends an asynchronous request to record the most recently reached checkpoint
 * on the backend.
 *
 * @param {string} stateName - The name of the state (checkpoint) that was reached.
 * @param {number} version - The version of the exploration at the time the checkpoint was
reached.
 *
 * @private
 */
function markCheckpointReached(stateName: string, version: number): void {}
```

ExplorationEngineService:

Service responsible for managing the core logic of exploration progression within the exploration player mode. This includes rendering HTML for exploration states and interactions, handling answer submission and classification, evaluating parameter values, managing state transitions, and coordinating media preloading (e.g., audio and images).

This service is mode-specific: it supports the runtime flow of *exploration-based* learning experiences. It does not handle broader conversation control logic, which is delegated to the ConversationFlowService.

Core responsibilities include:
- Rendering content and interactions for exploration states.
- Submitting and classifying learner answers using rule services.
- Updating exploration state based on learner responses and outcomes.
- Evaluating parameter values used for dynamic content rendering.
- Preloading and managing media resources tied to exploration content.

Note: This service is not responsible for general conversation orchestration or learner transcript logging. Those are handled by the ConversationFlowService and PlayerTranscriptService, respectively.

- What are the user interactions?
    - N/A
- Which states do I need?
    - All the states below are already maintained in the explorationEngineService, these are not new states:
        - currentStateName: string;
        - exploration: Exploration;
- What functions do we currently use in the conversation-skin?

```
/**
 * Returns the current state of the exploration based on
 * the last visited card in the transcript.
 *
 * @returns {State} The current state object.
 */
function getCurrentState(): State {}
```

```
/**
 * Retrieves and returns the state object corresponding to a given state name.
 *
 * @param {string} stateName - The name of the state to retrieve.
 * @returns {State} The state object for the provided state name.
 */
function getStateFromStateName(stateName: string): State {}
```

```
/**
 * Constructs a StateCard object for a given state name. It generates the
 * corresponding interaction and content HTML along with a unique random
 * suffix to ensure view refresh in Angular.
 *
 * Dependent state variables:
 * - this.exploration
 *
 * Dependent services:
 * - this.focusManagerService
 *
 * Side effects:
 * - Calls _getInteractionHtmlByStateName and _getRandomSuffix to build card content.
 *
 * @param {string} stateName - The name of the state to get the card for.
 * @returns {StateCard} A new StateCard object representing the state.
 */
getStateCardByName(stateName: string): StateCard { ... }
```

```
/**
 * Submits a learner's answer and computes the outcome based on interaction
 * rules, classification results, and exploration state transitions.
 * Also records answer stats, updates parameters, and prepares the next
 * state card for rendering.
 *
 * Dependent state variables:
 * - this.answerIsBeingProcessed
 * - this.nextStateName
 * - this.visitedStateNames
 * - this._editorPreviewMode
 *
 * Dependent services:
 * - this.exploration (used to fetch states, interactions)
```

```
 * - this.answerClassificationService (used for classifying the answer)
 * - this.statsReportingService (records submission metrics)
 * - this.learnerParamsService (fetches & updates exploration params)
 * - this.focusManagerService (generates focus label)
 * - this.alertsService (displays warnings)
 * - this.playerTranscriptService (manages transcript)
 *
 * Side effects:
 * - Emits `_updateActiveStateIfInEditorEventEmitter`
 * - Updates state tracking (`visitedStateNames`, `answerIsBeingProcessed`)
 * - May modify learner parameters
 * - May trigger alert messages
 *
 * @param {string} answer - The learner's submitted answer.
 * @param {InteractionRulesService} interactionRulesService - The service
 *     that contains interaction rules for answer classification.
 * @param {Function} successCallback - A callback invoked upon successfully
 *     handling the submitted answer. It is passed:
 *     - `nextCard` {StateCard}: The card representing the next state.
 *     - `refreshInteraction` {boolean}: Whether to refresh the UI interaction.
 *     - `feedbackHtml` {string}: The HTML string to display as feedback.
 *     - `refresherExplorationId` {string}: ID of refresher exploration, if any.
 *     - `missingPrerequisiteSkillId` {string}: ID of any missing skill, if any.
 *     - `remainOnCurrentCard` {boolean}: If the card should remain the same.
 *     - `taggedSkillMisconceptionId` {string}: Linked misconception ID (if any).
 *     - `wasOldStateInitial` {boolean}: True if old state is the init state.
 *     - `isFirstHit` {boolean}: True if first time reaching this state.
 *     - `isFinalQuestion` {boolean}: Always false for explorations.
 *     - `nextCardIfReallyStuck` {StateCard|null}: Card shown for stuck learners.
 *     - `focusLabel` {string}: Label for managing focus.
 *
 * @returns {boolean} Whether the submitted answer was correct.
 */
submitAnswer(
  answer: string,
  interactionRulesService: InteractionRulesService,
  successCallback: (
    nextCard: StateCard,
    refreshInteraction: boolean,
    feedbackHtml: string,
    refresherExplorationId: string,
    missingPrerequisiteSkillId: string,
    remainOnCurrentCard: boolean,
    taggedSkillMisconceptionId: string,
    wasOldStateInitial: boolean,
    isFirstHit: boolean,
    isFinalQuestion: boolean,
    nextCardIfReallyStuck: StateCard | null,
    focusLabel: string
  ) => void
): boolean { ... }
```

```
/**
 * Retrieves the list of author-recommended exploration IDs for a given state.
 *
 * @param {string} stateName - The name of the state.
 * @returns {string[]} A list of recommended exploration IDs.
 */
function getAuthorRecommendedExpIdsByStateName(stateName: string): string[] {}
```

```
/**
 * Computes the shortest path (by state transitions) from the initial state
 * of the exploration to the given destination state using BFS traversal.
 *
```

```
 * Dependent state variables:
 * - this.exploration.initStateName
 *
 * Dependent services:
 * - this.exploration (used to access states and transitions)
 *
 * Side effects:
 * - Builds internal maps and modifies temporary tracking structures.
 *
 * @param {StateObjectsBackendDict} allStates - All states in the exploration.
 * @param {string} destStateName - Name of the destination state.
 * @returns {string[]} List of state names forming the shortest path.
 */
getShortestPathToState(
  allStates: StateObjectsBackendDict,
  destStateName: string
): string[] { ... }
```

```
/**
 * Restarts the current lesson from the beginning. Clears any saved progress
 * and reinitializes the exploration state for the learner.
 *
 * Dependent state variables:
 * - this.explorationId (used to identify which exploration to reset)
 *
 * Dependent services:
 * - editableExplorationBackendApiService.resetExplorationProgressAsync()
 *   (sends a request to backend endpoint '/explorehandler/restart/' to clear saved progress)
 *
 * Side effects:
 * - Sends an HTTP request to reset the logged-in or logged-out learner's progress.
 * - Resets exploration progress on the server side.
 *
 * @function onRestartLesson
 * @returns {void}
 */
function restartLesson(): void {}
```

```
/**
 * Initializes the exploration player engine with the provided exploration
 * data, sets up services, and prepares the first card for rendering.
 * Also handles both preview and live modes.
 *
 * Dependent state variables:
 * - this.exploration
 * - this.answerIsBeingProcessed
 * - this.version
 * - this.visitedStateNames
 *
 * Dependent services:
 * - explorationObjectFactory (creates exploration instance)
 * - audioPreloaderService (handles preloading of audio)
 * - imagePreloaderService (handles preloading of images)
 * - entityTranslationsService (manages translation metadata)
 * - contentTranslationManagerService (sets original transcript language)
 * - contentTranslationLanguageService (initializes language preferences)
 *
 * Side effects:
 * - Initializes audio/image preloaders
 * - Updates internal state tracking
 * - Sets learner parameter values
 * - Triggers translation initialization
 * - May kick off auto-translation
 *
```

```
 * @param {ExplorationBackendDict} explorationDict - Backend dict of the exploration.
 * @param {number} explorationVersion - Version of the exploration.
 * @param {string | null} preferredAudioLanguage - Learner's preferred audio language.
 * @param {boolean} autoTtsEnabled - Whether auto text-to-speech is enabled.
 * @param {string[]} preferredContentLanguageCodes - Preferred content language codes.
 * @param {string[]} displayableLanguageCodes - All content language codes that can be shown.
 * @param {(stateCard: StateCard, label: string) => void} successCallback - Callback invoked
with the first state card and focus label once loading completes.
 *
 * @returns {void}
 */
init(
  explorationDict: ExplorationBackendDict,
  explorationVersion: number,
  preferredAudioLanguage: string | null,
  autoTtsEnabled: boolean,
  preferredContentLanguageCodes: string[],
  displayableLanguageCodes: string[],
  successCallback: (stateCard: StateCard, label: string) => void
): void { ... }
```

PlayerPositionService:

Service for keeping track of the learner's position.

- What are the user interactions?
    - Move one card forward
    - Move one card backward
- Which states do I need?
    - displayedCardIndex!: number;
    - onChangeCallback!: Function;
- What functions do we currently use in the conversation skin?

```
/**
 * Called to move the user to the next card in the sequence.
 */
function onMoveToNewCard(): void {}
```

```
/**
 * Called to move the user to the previous card in the sequence.
 */
function onMoveToPreviousCard(): void {}
```

```
/**
 * Gets the index of the currently displayed card.
 *
 * @returns {number} The index of the displayed card.
 */
function getDisplayedCardIndex(): number {}
```

```
/**
 * init
 * Initializes the service with a provided callback and resets the displayed card index.
 * The callback is stored for later execution on relevant state changes.
 *
 * Dependent state variables:
 * - this.displayedCardIndex
 * - this.onChangeCallback
 *
```

```
 * Dependent services:
 * - None
 *
 * Side effects:
 * - Resets the card index to -1.
 * - Stores the callback function for later use.
 *
 * @function init
 * @param {Function} callback - The function to call when the displayed card index changes.
 * @returns {void}
 */
init(callback: Function): void {
  this.displayedCardIndex = -1;
  this.onChangeCallback = callback;
}
```

```
/**
 * Retrieves the name of the current state being displayed.
 *
 * @returns {string} The name of the current state.
 */
function getCurrentStateName(): string {}
```

```
/**
 * Sets the index of the currently displayed card.
 *
 * @param {number} index - The index to set as currently displayed.
 * @throws Will throw an error if the onChange callback is not initialized.
 */
function setDisplayedCardIndex(index: number): void {}
```

ExplorationPlayerModeService

Service for keeping the track of the mode of the exploration player (exploration, pretest, question player, diagnostic test, and story chapter).

- What are the user interactions?
    - N/A
- Which states do I need?
    - currentMode: string (enum)

```
EXPLORATION_MODE: {
  DIAGNOSTIC_TEST_PLAYER: 'diagnostic_test_player',
  EXPLORATION: 'exploration',
  PRETEST: 'pretest',
  QUESTION_PLAYER: 'question_player',
  STORY_CHAPTER: 'story_chapter',
}
```

    - PageIsIframed: boolean;
- What functions do we currently use in the conversation skin?

```
/**
 * Determines whether the current mode is Question Player mode.
 *
```

```
 * @returns {boolean} True if in Question Player mode, false otherwise.
 */
function isInQuestionPlayerMode(): boolean {}
```

```
/**
 * Determines whether the current mode is either Pretest or Question Player mode.
 *
 * @returns {boolean} True if in question-related mode, false otherwise.
 */
function isInQuestionMode(): boolean {}
```

```
/**
 * Determines whether the current mode is Diagnostic Test Player mode.
 *
 * @returns {boolean} True if in Diagnostic Test Player mode, false otherwise.
 */
function isInDiagnosticTestPlayerMode(): boolean {}
```

```
/**
 * Determines whether the player is currently in Story Chapter mode.
 *
 * @returns {boolean} True if in Story Chapter mode, false otherwise.
 */
function isInStoryChapterMode(): boolean {}
```

```
/**
 * Determines if the player is currently presenting only isolated questions,
 * i.e., in modes where only questions are shown without additional learning content.
 * These modes include Question Player, Diagnostic Test Player, and Pretest.
 * Exploration and Story Chapter modes include learning content along with questions.
 *
 * @throws {Error} Throws an error if the exploration mode is invalid.
 * @returns {boolean} True if presenting isolated questions, false otherwise.
 */
function isPresentingIsolatedQuestions(): boolean {}
```

```
/**
 * Initializes the exploration context by parsing the URL and
 * determining whether the user is in exploration, editor,
 * skill editor, embed, or lesson mode.
 */
init(): void {}
```

StateEditorService:

Service for managing the state editor in the exploration editor. Maintains internal state such as:
- Active state name and interaction configuration
- Linked skill ID, misconceptions, and solution validity
- UI initialization flags and form validity

Provides functions to:
- Get/set interaction and state-level data
- Track editor initialization status
- Emit events for UI updates and validation

- What are the user interactions?
    - N/A
- Which states do I need?
    - private _updateActiveStateIfInEditorEventEmitter: EventEmitter<string> = new EventEmitter();

- What functions do we currently use in the conversation skin?

```
/**
 * Returns an event emitter that emits the active state name when in editor mode. This event is
 * primarily used to ensure that the active state remains consistent when navigating
 * through preview mode and switching back to the editor.
 *
 * Subscribing to this event allows components to respond to state changes specifically in the
 * editor context without interfering with preview mode navigation.
 *
 * @returns {EventEmitter<string>} An event emitter that emits the name
 * of the active state to update when in editor mode.
 */
function get onUpdateActiveStateIfInEditor(): EventEmitter<string> {}
```

CurrentInteractionService:

Facilitates communication between the current interaction and the progress nav. The former holds data about the learner's answer, while the latter contains the actual "Submit" button which triggers the answer submission process.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
 * Registers the onSubmit callback function for the current interaction.
 *
 * This function allows the current interaction (e.g., multiple choice,
 * text input, etc.) to specify what should happen when the learner submits
 * an answer. The function must accept the answer and an InteractionRulesService
 * to classify and evaluate it.
 *
 * This should be called by the interaction component itself (usually from
 * ConversationSkinDirective or similar components) during setup.
 *
 * @param {OnSubmitFn} onSubmit - A function that handles answer submission logic.
 */
function setOnSubmitFn(onSubmit: OnSubmitFn): void {}
```

```
/**
 * Determines whether the "Submit" button in the UI should be disabled.
 *
 * The button is disabled if the interaction has not registered a submission
 * function (e.g., due to slow loading in low-bandwidth environments), or if
 * the interaction has a validity check function and it returns false.
 *
 * Interactions may define a `_validityCheckFn` to determine whether the
 * current answer input is in a valid state. If no validity check function
 * is defined, the answer is assumed valid and the submit button is enabled.
 *
 * @returns {boolean} True if the submit button should be disabled, false otherwise.
 */
function isSubmitButtonDisabled(): boolean {}
```

```
/**
 * Clears all pre-submit hooks for the current interaction.
```

```
 *
 * Pre-submit hooks are callback functions registered by interactions
 * that should be executed right before the learner submits an answer
 * (e.g., validation, logging, or UI changes).
 *
 * This function should be called every time a new card is loaded to ensure
 * that old hooks from previous interactions are removed.
 */
function clearPresubmitHooks(): void {}
```

```
/**
 * Initiates the answer submission process for the current interaction.
 *
 * This method is typically triggered when the learner clicks the "Submit" button.
 * If the interaction has not registered a `_submitAnswerFn`, an error is thrown
 * with detailed context for debugging (e.g., interaction ID, state name, etc.).
 *
 * Otherwise, it invokes the registered submission function, which is responsible
 * for classifying the answer, generating feedback, and transitioning to the
 * next state.
 *
 * @throws {Error} If the interaction did not register a submit function before submission.
 */
function submitAnswer(): void {}
```

PageContextService:

Service for returning information about a page's context.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
 * Determines whether the application is currently in preview mode.
 *
 * @returns {boolean} True if in preview mode, false otherwise.
 */
function isInPreviewMode(): boolean {}

Moved from explorationPlayerEngineService
```

```
/**
 * Checks whether the current page context is the Exploration Editor.
 *
 * This utility method is typically used to conditionally execute logic
 * that should only run within the exploration editor (as opposed to
 * the player, question editor, etc.).
 *
 * It compares the current page context against the constant
 * `EXPLORATION_EDITOR` defined in `ServicesConstants`.
 *
 * @returns {boolean} True if the current context is the Exploration Editor, false otherwise.
 */
function isInExplorationEditorPage(): boolean {}
```

ExplorationRecommendationsService:

Service for recommending explorations at the end of an exploration.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
 * Fetches a list of recommended exploration summaries for the learner.
 *
 * This method retrieves exploration summaries based on:
 * - Author-specified recommendations.
 * - Optional system-generated recommendations (if enabled and not in the editor).
 * - Contextual information like collection ID, story ID, and node ID, if present in the URL.
 *
 * System recommendations are excluded if the current page is the editor or if
 * `includeAutogeneratedRecommendations` is false.
 *
 * The recommendations are retrieved asynchronously from the backend API, and
 * the result is passed to the provided `successCallback`.
 *
 * @param {string[]} authorRecommendedExpIds - Array of exploration IDs recommended by the
content author.
 * @param {boolean} includeAutogeneratedRecommendations - Whether to include system-generated
recommendations.
 * @param {(value: LearnerExplorationSummary[]) => void} successCallback - A callback function
invoked with the resulting list of summaries.
 */
function getRecommendedSummaryDicts(
  authorRecommendedExpIds: string[],
  includeAutogeneratedRecommendations: boolean,
  successCallback: (value: LearnerExplorationSummary[]) => void
): void {}
```

FatigueDetectionService:

Service for detecting spamming behavior from the learner.

- What are the user interactions?
    - N/A
- Which states do I need?
    - private submissionTimesMsec: number[] = [];
    - private windowStartTime!: number;
    - private windowEndTime!: number;
- What functions do we currently use in the conversation skin?

```
/**
 * Records the current timestamp (in milliseconds) when a learner submits an answer.
 *
 * This timestamp is stored in an internal list and is later used to detect
 * rapid, repeated submissions that may indicate spamming behavior.
 */
function recordSubmissionTimestamp(): void {}
```

```
/**
 * Determines whether the learner is submitting answers too rapidly.
 *
 * It uses a sliding window approach: if a number of submissions exceeds the
```

```
 * `SPAM_COUNT_THRESHOLD` within a short time window (`SPAM_WINDOW_MSEC`),
 * the learner is considered to be submitting too fast.
 *
 * @returns {boolean} True if submissions are too fast and spamming is detected; false
otherwise.
 */
function isSubmittingTooFast(): boolean {}
```

```
/**
 * Displays a modal asking the learner to take a break.
 *
 * This modal is triggered when spamming behavior is detected.
 * It prevents further interaction until the learner dismisses it.
 *
 * The modal is static (cannot be closed by clicking outside of it).
 */
function displayTakeBreakMessage(): void {}
```

```
/**
 * Resets the submission timestamps.
 *
 * This is typically called after a timeout, modal dismissal, or when
 * answer tracking needs to be cleared to start fresh.
 */
function reset(): void {}
```

FocusManagerService:

Service for setting focus. This broadcasts a 'focusOn' event which sets focus to the element in the page with the corresponding focusOn attribute. This requires LABEL_FOR_CLEARING_FOCUS to exist somewhere in the HTML page.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
 * Sets focus to a specific element if the learner is using a desktop device.
 *
 * This avoids setting focus on mobile devices, where autofocus behavior
 * can be disruptive (e.g., automatically opening the on-screen keyboard).
 *
 * @param {string} newFocusLabel - The unique label used to identify the DOM element to focus.
 */
function setFocusIfOnDesktop(newFocusLabel: string): void {}
```

```
/**
 * Generates a new, unique focus label.
 *
 * This label is typically used to programmatically set focus on input elements
 * during dynamic UI updates. Internally, it delegates to the ID generation service.
 *
 * @returns {string} A unique string label that can be used for focus tracking.
 */
function generateFocusLabel(): string {}
```

HintsAndSolutionManagerService:

Service responsible for managing the release, display, and consumption of hints and solutions during a learner's session.

This includes triggering time-based hint availability, showing tooltips, tracking the learner's usage of hints and solutions, and emitting events when key hint/solution milestones are reached.

Core responsibilities include:
- Managing the timing and visibility of hints and the solution.
- Tracking which hints have been released and consumed.
- Emitting events when all hints are exhausted or when the learner appears stuck.
- Controlling tooltip behavior for hint/solution icons.

Note: This service does not log user actions (Like number of answers submitted) — that is handled by the PlayerTranscriptService.

- What are the user interactions?
    - N/A
- Which states do I need?
    - numHintsReleased: number = 0;
    - solutionReleased: boolean = false;
    - hintsForLatestCard: Hint[] = [];
    - correctAnswerSubmitted: boolean = false;
- What functions do we currently use in the conversation skin?

```
/**
 * Event emitted when the learner views the solution.
 *
 * This is typically triggered after the learner clicks to reveal the solution
 * once it's available. Listeners can use this to log analytics or update the UI.
 *
 * @returns {EventEmitter<void>} Emits when the solution is viewed.
 */
get onSolutionViewedEventEmitter(): EventEmitter<void> {}
```

```
/**
 * Event emitted when a hint is consumed (viewed) by the learner.
 *
 * This event can be used to track when a learner opens a hint,
 * which may inform pacing or adaptive feedback mechanisms.
 *
 * @returns {EventEmitter<void>} Emits when a hint is viewed.
 */
get onHintConsumed(): EventEmitter<void> {}
```

```
/**
 * Event emitted when all available hints for the current question have been used.
 *
 * Useful for triggering logic such as starting a timer before showing the solution,
 * or for UX/UI indicators that no further hints are available.
 *
 * @returns {EventEmitter<string>} Emits the current state name when all hints are exhausted.
 */
get onHintsExhausted(): EventEmitter<string> {}
```

```
/**
 * Event emitted when the learner is considered "really stuck".
 *
 * This typically occurs after the learner has consumed all hints and submitted
 * multiple incorrect answers without progress. It can be used to trigger
 * fallback actions like showing the solution automatically or offering help.
 *
 * @returns {EventEmitter<string>} Emits the current state name when the learner is stuck.
 */
get onLearnerReallyStuck(): EventEmitter<string> {}
```

```
/**
 * Records that the learner has submitted a wrong answer.
 *
 * This method adjusts the internal counters used to control hint and solution behavior:
 * - If a hint is currently waiting to be viewed, it does nothing.
 * - If hints are available and not all are exhausted, it tracks wrong answers to
 *   determine when to accelerate the release of the next hint.
 * - If all hints have been exhausted and the learner continues submitting wrong answers,
 *   it tracks the number of post-hint wrong attempts. After a threshold (typically 3 total),
 *   it emits the `onLearnerReallyStuck` event to signal that the learner may need additional
help.
 *
 */
function recordWrongAnswer(): void {}
```

```
/**
 * Releases the solution for the current question to the learner.
 *
 * This method performs the following actions:
 * - Marks the solution as released by setting `solutionReleased` to true.
 * - If the learner hasn't already discovered the solution and the tooltip timer
 *   hasn't started, it sets a timeout to show the solution tooltip after a fixed delay.
 * - Emits the `_timeoutElapsedEventEmitter` to notify any subscribers that the
 *   solution release timer has elapsed, which may trigger additional UI updates
 *   or logging events.
 *
 * This function supports delayed solution reveal, ensuring learners have a moment
 * to process the situation before the solution tooltip is shown, improving UX.
 */
function releaseSolution(): void {}
```

I18nLanguageCodeService:

Service for managing the i18n language code used across Oppia's user interface. This includes broadcasting language changes, providing utility functions for determining language direction (LTR/RTL), and generating translation keys for various entity types (e.g., topics, subtopics, classrooms, stories, explorations).

This service also supports "hacky" translation keys used during transitional phases before full translation dashboard support is available.

Note: Some parts of the service are intended to be temporary and are marked with TODOs for future cleanup once the translation infrastructure is complete.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A

- What functions do we currently use in the conversation skin?

```
 /**
   * Takes exploration id and entity translationKey type as input, generates
   * and returns the translation key based on that.
   * @param {string} explorationId - Unique id of the exploration, used to
   * generate translation key.
   * @param {TranslationKeyType} keyType - either Title or Description.
   * @returns {string} - translation key for the exploration name/description.
   */
  getExplorationTranslationKey(
    explorationId: string,
    keyType: TranslationKeyType
  ): string {}
```

```
/**
 * Sets the current internationalization (i18n) language code.
 * Updates the previous language code, assigns the new code, and emits an event
 * notifying other parts of the app about the language change.
 *
 * @param {string} code - The new language code to set.
 */
function setI18nLanguageCode(code: string): void {}
```

```
 /**
   * Checks if the translation key is valid by checking if it is present
   * in the constants file which indicates that the translation key is
   * present in the language JSON file.
   * @param {string} translationKey - The translation key whose existence in
   * the language JSON file needs to be checked.
   * @returns {boolean} - Whether the given translation key is present
   * in the language JSON files.
   */
  isHackyTranslationAvailable(translationKey: string): boolean {}
```

```
/**
 * Checks if the current internationalization language code is English.
 *
 * @return {boolean} True if the current language code is 'en', false otherwise.
 */
function isCurrentLanguageEnglish(): boolean {}
```

LearnerAnswerInfoService:

Service for managing the learner answer info feature. This service determines whether the learner should be asked for more information about their answer, based on various criteria such as:
- Whether the state is configured to solicit answer details
- The type of outcome (default, correct, or general)
- A randomized probability mechanism

Note: This feature helps improve the quality of learner analytics by optionally collecting qualitative reasoning behind user-submitted answers.

- What are the user interactions?
    - N/A
- Which states do I need?
    - private currentEntityId!: string;
    - private interactionId!: string | null;
    - private currentAnswer!: string;

    - private currentInteractionRulesService!: InteractionRulesService;
    - private submittedAnswerInfoCount: number = 0;
    - private canAskLearnerForAnswerInfo: boolean = false;
    - private probabilityIndexes = {}
- What functions do we currently use in the conversation skin?

```
/**
 * Returns whether the system can currently ask the learner for additional answer details.
 *
 * @return {boolean} True if learner can be asked for answer info, false otherwise.
 */
function getCanAskLearnerForAnswerInfo(): boolean {}
```

```
/**
 * Initializes the learner answer info service with the current interaction and answer details.
 * Determines if the learner should be prompted for more information based on interaction state,
 * answer classification, and configured probabilities.
 *
 * @param {string} entityId - The ID of the current entity (e.g., exploration or question).
 * @param {State} state - The current state of the interaction.
 * @param {string} answer - The learner's submitted answer.
 * @param {InteractionRulesService} interactionRulesService - Service for interaction rules.
 * @param {boolean} alwaysAskLearnerForAnswerInfo - If true, always prompt the learner.
 */
function initLearnerAnswerInfoService(
  entityId: string,
  state: State,
  answer: string,
  interactionRulesService: InteractionRulesService,
  alwaysAskLearnerForAnswerInfo: boolean
): void {}
```

```
/**
 * Generates the HTML string for the localized question asking the learner for answer details.
 *
 * @return {string} The HTML string with the translated solicit answer details question.
 */
function getSolicitAnswerDetailsQuestion(): string {}
```

LocalStorageService:

Utility service for saving data locally on the client machine.

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
 * Removes the unique progress ID of a logged-out learner from local storage.
 *
 * This helps reset progress tracking when it's no longer needed.
 */
function removeUniqueProgressIdOfLoggedOutLearner(): void {}
```

```
/**
 * Retrieves the unique progress ID for a logged-out learner from local storage.
 *
```

```
 * @returns {string | null} The stored unique progress ID, or null if unavailable.
 */
function getUniqueProgressIdOfLoggedOutLearner(): string | null {}
```

PlayerTranscriptService:

Note: Merge NumberAttemptsService with it

Service responsible for tracking the full transcript of a learner's session. This includes recording the sequence of cards shown, answers submitted, feedback provided, and interactions with hints and solutions. This log supports features such as displaying previous cards, restoring session state, and analyzing learner progression.

Core responsibilities include:
- Recording each card presented to the learner.
- Logging answers, feedback, and correctness.
- Tracking hint/solution consumption events.
- Providing access to the current and past cards in the transcript.

Note: This service is purely for data storage and retrieval. It does not manage interaction flow or hint logic — those are handled by the engine and hint manager services, respectively.

- What are the user interactions?
  - N/A
- Which states do I need?
  - numberAttempts (Merge from NumberAttemptsService)
  - visitedStateNames: string[] = [];
  - solutionConsumed: boolean = false;
  - wrongAnswersSinceLastHintConsumed: number = 0;
  - wrongAnswersAfterHintsExhausted: number = 0;
  - numHintsConsumed: number = 0;
  - transcript: StateCard[] = [];
  - numAnswersSubmitted = 0;
- What functions do we currently use in the conversation skin?

```
/**
 * Returns the number of cards currently in the transcript.
 *
 * @returns {number} Total number of cards in the transcript.
 */
function getNumCards(): number {}
```

```
/**
 * Retrieves the farthest state name that a user has found.
 *
 * @returns {string} Name of the last state.
 */
function getLastStateName(): string {}
```

```
/**
 * Checks if the given index corresponds to the last card.
 *
 * @param {number} index - Index to check.
 * @returns {boolean} True if index is for the last card.
 */
function isLastCard(index: number): boolean {}
```

```
/**
 * Checks whether the learner has visited the specified state before.
 *
 * @param {string} stateName - Name of the state to check.
 * @returns {boolean} True if the state was encountered before.
 */
function hasEncounteredStateBefore(stateName: string): boolean {}
```

```
/**
 * Finds the index of the latest card with a given state name.
 *
 * @param {string} name - The state name to search for.
 * @returns {number | null} Index of the latest matching card or null.
 */
function findIndexOfLatestStateWithName(name: string): number | null {}
```

```
/**
 * Adds a duplicate of the previous card to the transcript, marked incomplete.
 *
 * Throws an error if the current card is the first card.
 */
function addPreviousCard(): void {}
```

```
/**
 * Adds a new learner input to the latest card in the transcript.
 *
 * Throws an error if input is added before receiving a response.
 * @param {string} input - Learner's input.
 * @param {boolean} isHint - Whether the input was a hint.
 */
function addNewInput(input: string, isHint: boolean): void {}
```

```
/**
 * Updates the interaction HTML of the latest card.
 *
 * @param {string} newInteractionHtml - The new HTML to display.
 */
function updateLatestInteractionHtml(newInteractionHtml: string): void {}
```

```
**
 * Records an answer submission for the given question.
 * If the solution was already viewed, correct answers are not recorded.
 *
 * @param {Question} question - The question answered.
 * @param {boolean} isCorrect - Whether the submitted answer was correct.
 * @param {string} taggedSkillMisconceptionId - ID of tagged misconception, if any.
 */
function recordAnswer(
  question: Question,
  isCorrect: boolean,
  taggedSkillMisconceptionId: string
): void {}

Moved from QuestionPlayerStateService
```

```
/**
 * Records that a hint was used for the given question.
 * Initializes tracking data if it does not exist.
 *
 * @param {Question} question - The question for which the hint was used.
 */
function useHint(question: Question): void {}
```

```
Moved from QuestionPlayerStateService
```

```
/**
 * Records that the solution was viewed for the given question.
 * Initializes tracking data if it does not exist.
 *
 * @param {Question} question - The question for which the solution was viewed.
 */
function viewSolution(question: Question): void {}

Moved from QuestionPlayerStateService
```

```
/**
 * Records that a new card has been added by updating the current state name
 * to the next state's name.
 */
function recordNewCardAdded(): void {}

Moved from ExplorationPlayerEngineService
```

QuestionPlayerEngineService:

Service responsible for managing the core logic of the question player used in practice sessions. This service handles functionality such as question initialization, answer submission, feedback evaluation, and navigation between questions based on learner input.

Core responsibilities include:
- Initializing and randomizing a list of question objects.
- Rendering questions and interactions using HTML formatting.
- Processing and classifying learner answers via rule services.
- Displaying feedback and advancing through the question list.

- What are the user interactions?
    - N/A
- Which states do I need?
    - private questions: Question[] = [];
    - private currentIndex: number = null;
    - private nextIndex: number = null;
- What functions do we currently use in the conversation skin?

```
/**
 * Initializes the question player with a randomized list of questions.
 * Sets up the first question card and prepares the context.
 *
 * Dependent state variables:
 * - this.questions
 * - this.currentIndex
 *
 * Dependent services:
 * - ContextService
 * - AlertsService
 *
 * Side effects:
 * - Shuffles and stores the questions.
 * - Calls loadInitialQuestion which triggers successCallback or errorCallback.
 * - Updates context to indicate question player is open.
 *
 * @param {Question[]} questionObjects - Array of questions to be played.
```

```
 * @param {function} successCallback - Called with initial card and focus label on success.
 * @param {function=} errorCallback - Called on initialization failure.
 */
init(
  questionObjects: Question[],
  successCallback: (initialCard: StateCard, nextFocusLabel: string) => void,
  errorCallback?: () => void
): void {}
```

```
/**
 * Returns the current question object being played.
 *
 * Dependent state variables:
 * - this.questions
 * - this.currentIndex
 *
 * Dependent services:
 * - None
 *
 * Side effects:
 * - None
 *
 * @returns {Question} The current Question object.
 */
getCurrentQuestion(): Question {}
```

```
/**
 * Clears the list of loaded questions by resetting the questions array, e.g. when the
 * skill-preview-tab component ‘s selectQuestionToPreview() makes a call to this
 * function.
 *
 * Dependent state variables:
 * - this.questions
 *
 * Dependent services:
 * - None
 *
 * Side effects:
 * - Empties the questions array.
 *
 * @returns {void}
 */
clearQuestions(): void {}
```

```
/**
 * TODO: StatsReportingService should figure out the language code from the user
 * language instead of from the current question. (Refactor)
 *
 * Returns the language code of the current question.
 *
 * Dependent state variables:
 * - this.questions
 * - this.currentIndex
 *
 * Dependent services:
 * - None
 *
 * Side effects:
 * - None
 *
 * @returns {string} Language code of the current question.
 */
getLanguageCode(): string
```

```
/**
 * Processes a submitted answer, classifies it, generates feedback,
 * determines the next question or whether to stay on the current one,
 * and calls the success callback with all relevant info.
 *
 * Dependent state variables:
 * - this.currentIndex
 * - this.questions
 *
 * Dependent services:
 * - AnswerClassificationService
 * - ExpressionInterpolationService
 * - ExplorationHtmlFormatterService
 * - FocusManagerService
 * - AlertsService
 *
 * Side effects:
 * - Potentially updates this.nextIndex
 * - Calls successCallback with parameters for UI update
 *
 * @param {InteractionAnswer} answer - The user's submitted answer.
 * @param {InteractionRulesService} interactionRulesService - Rules to classify the answer.
 * @param {function} successCallback - Callback invoked with next card info and feedback.
 * @returns {boolean|undefined} True if the answer is correct; undefined if processing was
blocked.
 */
submitAnswer(
  answer: InteractionAnswer,
  interactionRulesService: InteractionRulesService,
  successCallback: (
    nextCard: StateCard,
    refreshInteraction: boolean,
    feedbackHtml: string,
    refresherExplorationId,
    missingPrerequisiteSkillId,
    remainOnCurrentCard: boolean,
    taggedSkillMisconceptionId: string,
    wasOldStateInitial,
    isFirstHit,
    isFinalQuestion: boolean,
    nextCardIfReallyStuck: null,
    focusLabel: string
  ) => void
): boolean | undefined {}
```

DiagnosticPlayerEngineService:

Engine service responsible for managing the core logic of diagnostic test progression. This includes fetching questions, handling answer submission, transitioning between topics and skills, and tracking learner progress. The service coordinates test flow across multiple topics and determines when the diagnostic session is complete. It delegates UI rendering, recommendation logic, and analytics context to higher-level services.

Core responsibilities include:
- Initializing the test session from topic tracking models.
- Fetching and presenting the next question based on learner performance.
- Submitting and classifying answers using the rules and outcome services.
- Transitioning between skills and topics based on correctness.
- Emitting progress and completion events to inform the player shell.
- Tracking attempted questions and preventing duplicates.

Note: This service is specific to the diagnostic test mode and is called by the ConversationFlowService. It does not handle UI rendering directly or transcript logging.

- What are the user interactions?
    - N/A
- Which states do I need?
    - private _diagnosticTestTopicTrackerModel!: DiagnosticTestTopicTrackerModel;
    - private _initialCopyOfTopicTrackerModel!: DiagnosticTestTopicTrackerModel;
    - private _diagnosticTestCurrentTopicStatusModel!: DiagnosticTestCurrentTopicStatusModel;
    - private _currentQuestion!: Question;
    - private _currentTopicId!: string;
    - private _currentSkillId!: string;
    - private _numberOfAttemptedQuestions!: number;
    - private _focusLabel!: string;
    - private _encounteredQuestionIds!: string[];
- What functions do we currently use in the conversation skin?

```
/**
 * Initializes the diagnostic test engine with the given topic tracker model.
 * It selects the first topic to test and fetches questions for it using the
 * backend API. After retrieving the questions, it sets the current topic,
 * skill, and question, and then triggers the success callback with a state card.
 *
 * Dependent state variables:
 * - _diagnosticTestTopicTrackerModel
 * - _initialCopyOfTopicTrackerModel
 * - _currentTopicId
 * - _currentSkillId
 * - _currentQuestion
 * - _diagnosticTestCurrentTopicStatusModel
 * - _encounteredQuestionIds
 *
 * Dependent services:
 * - questionBackendApiService
 * - alertsService
 *
 * Side effects:
 * - Sets internal state for the current topic, skill, and question.
 * - Calls the backend API to fetch diagnostic test questions.
 * - Triggers successCallback with initial StateCard and focus label.
 * - Emits a warning via alertsService if the API fails.
 *
 * @function init
 * @param {DiagnosticTestTopicTrackerModel} diagnosticTestTopicTrackerModel - The model tracking
topic progress.
 * @param {(initialCard: StateCard, nextFocusLabel: string) => void} successCallback - Callback
to continue the test flow.
 * @returns {void}
 */
init(
  diagnosticTestTopicTrackerModel: DiagnosticTestTopicTrackerModel,
  successCallback: (initialCard: StateCard, nextFocusLabel: string) => void
): void {}
```

```
/**
 * Processes the user's submitted answer, classifies it,
 * records the attempt as correct or incorrect,
 * retrieves the next question asynchronously,
 * and calls the provided success callback with the results.
```

```
 * Emits progress or completion events depending on the state.
 *
 * Dependent state variables:
 * - _currentQuestion
 * - _numberOfAttemptedQuestions
 * - _currentSkillId
 * - _focusLabel
 *
 * Dependent services:
 * - answerClassificationService
 * - diagnosticTestCurrentTopicStatusModel
 * - diagnosticTestPlayerStatusService
 *
 * Side effects:
 * - Updates attempt count and topic status
 * - Emits session progress and completion events
 * - Calls successCallback with detailed feedback and next card info
 *
 * @function submitAnswer
 * @param {InteractionAnswer} answer - The submitted answer.
 * @param {InteractionRulesService} interactionRulesService - Service to evaluate rules.
 * @param {Function} successCallback - Callback invoked with detailed parameters on success.
 * @returns {boolean} True if the submitted answer is correct, else false.
 */
submitAnswer(
  answer: InteractionAnswer,
  interactionRulesService: InteractionRulesService,
  successCallback: (
    nextCard: StateCard,
    refreshInteraction: boolean,
    feedbackHtml: string,
    feedbackAudioTranslations: BindableVoiceovers,
    refresherExplorationId: string,
    missingPrerequisiteSkillId: string,
    remainOnCurrentCard: boolean,
    taggedSkillMisconceptionId: string,
    wasOldStateInitial: boolean,
    isFirstHit: boolean,
    isFinalQuestion: boolean,
    nextCardIfReallyStuck: StateCard,
    focusLabel: string
  ) => void
): boolean {}
```

```
/**
 * Records an incorrect attempt for the current skill,
 * retrieves the next question asynchronously,
 * and calls the provided callback with the new StateCard.
 * Emits progress or completion events depending on the state.
 *
 * Dependent state variables:
 * - _currentSkillId
 *
 * Dependent services:
 * - diagnosticTestCurrentTopicStatusModel
 * - diagnosticTestPlayerStatusService
 *
 * Side effects:
 * - Updates topic status
 * - Emits session progress and completion events
 * - Calls successCallback with the next StateCard
 *
 * @function skipCurrentQuestion
 * @param {(stateCard: StateCard) => void} successCallback - Callback invoked with the next
StateCard.
```

```
 * @returns {void}
 */
skipCurrentQuestion(successCallback: (stateCard: StateCard) => void): void {}
```

```
/**
 * Returns the language code of the next question for the current skill.
 *
 * Dependent state variables:
 * - _diagnosticTestCurrentTopicStatusModel
 * - _currentSkillId
 *
 * Dependent services:
 * - None
 *
 * Side effects:
 * - None
 *
 * @function getLanguageCode
 * @returns {string} The language code of the next question.
 */
getLanguageCode(): string {}
```

UrlService:

Service for extracting, constructing, and manipulating components of the current page URL. Provides utility methods to:

- Parse query parameters and hash fragments
- Extract identifiers (e.g., topic ID, story ID, skill ID) from URLs
- Generate URLs with query parameters

- What are the user interactions?
    - N/A
- Which states do I need?
    - N/A
- What functions do we currently use in the conversation skin?

```
/**
  * This function is used to find the collection id from
  * the exploration url.
  * @return {string} a collection id if
  * the url parameter doesn't have a 'parent' property
  * but have a 'collection_id' property; @return {null} if otherwise.
  */
 getCollectionIdFromExplorationUrl(): string | null {}
```

```
/**
  * This function is used to check whether the url is framed.
  * @return {boolean} whether the url is framed.
  */
 isIframed(): boolean {}

TODO: Remove it from here
```

```
/**
  * This function is used to find the current path name.
  * @return {string} the current path name.
  */
 getPathname(): string {}
```

```
/**
  * This function is used to find the topic URL fragment
  * from the learner's url.
  * @return {string} the topic URL fragment.
  * @throws Will throw an error if the url is invalid.
  */
 getTopicUrlFragmentFromLearnerUrl(): string {}
```

```
/**
  * This function is used to combine the url, the field name,
  * and the field value together.
  * @param {string} url - the url.
  * @param {string} fieldName - the field name.
  * @param {string} fieldValue - the field value.
  * @return {string[]} the list of query field values.
  */
 addField(url: string, fieldName: string, fieldValue: string): string {}
```

```
/**
  * This function is used to find the query values as a list.
  * @param {string} fieldName - the name of the field.
  * @return {string[]} the list of query field values.
  */
 getQueryFieldValuesAsList(fieldName: string): string[] {}
```

```
/**
  * This function is used to find the query values as a list.
  * @param {string} fieldName - the name of the field.
  * @return {string[]} the list of query field values.
  */
 getQueryFieldValuesAsList(fieldName: string): string[] {
```

```
/**
 * Parses the current page's query string and returns URL parameters.
 *
 * @returns {UrlParamsType} An object mapping parameter names to values.
 */
getUrlParams(): UrlParamsType { ... }
```

```
/**
 * Retrieves the story URL fragment from the learner or exploration URL.
 * Handles both learner view and exploration player routes.
 *
 * @returns {string|null} The story URL fragment or null if not found.
 */
getStoryUrlFragmentFromLearnerUrl(): string | null { ... }
```

```
/**
 * Extracts the classroom URL fragment from the learner or exploration URL.
 * Throws an error if not found or the URL is invalid.
 *
 * @returns {string} The classroom URL fragment.
 * @throws {Error} If the URL is invalid for determining classroom fragment.
 */
getClassroomUrlFragmentFromLearnerUrl(): string { ... }
```

Future Work (Not in scope)

- Implementation of tutorial for the lesson player
- Implementation of Restart tutorial in the help modal

- Implementation of the audio speed adjuster
- As we are removing the lesson-language change option and combining it with the site language, Suppose, if the selected site language is French and that lesson has not been translated into french yet, so we will be showing it in the English as the default language, but I think we should some kind of icon or something on the lesson player at the top corner, to avoid learners getting confused about this thing.

Key User Stories (Future Work)

#	**Title**	**User Story Description (role, goal, motivation)** *"As a …, I need …, so that …."*	**List of tasks needed to achieve the goal (this is the "User Journey")**	**Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.**
1	Onboarding Lesson player tour	**As a** new Learner, **I should** be offered a full usage tour of the lesson player **so that** I can know how to use the lesson player.	Visit any lesson Click the "Yes, please" button in the modal View the tooltip Click on the "next" or "got it" button when done	This is for only new users or logged-out users. Modal Mock Tooltip Mock
2	See the detailed lesson info (Help Button)	**As a** Learner, **I should be** able to view the details of the lesson **so that** I can get to know about the analytics of the lesson.	Visit any lesson Click on the "Help" button in the sidebar Help modal pops up Click on the "Lesson Information" button Lesson info modal pops up View the details of the lessons	Help modal mock Lesson info mock
	See the tutorial (Help Button)	**As a** Learner, **I should be** able to view the tutorial for the lesson player whenever I want **so that** I can effectively use the lesson player.	Visit any lesson Click on the "Help" button in the sidebar Help modal pops up Click on the "Restart tutorial tips" button View the tooltip Click on the "next" or "got it" button when done	Help modal mock

3	Change the speed of the lesson audio	**As a** Learner, **I should be** able to change the speed of the lesson voiceover **so that** I can listen to the lesson voiceover at my pace.	Visit any lesson	Audio bar mock
			Click on the speed button and set the appropriate speed.	
			Voiceover audio will be resumed from that position at the selected speed.	

UI Screens/Components (Future Work)

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	Help modal	The help modal will provide detailed lesson information. (It will also be used to revisit the tooltip)	Yes	Mock	Same as above Implement a close button with a clear label (`aria-label="Close Help Modal"`)